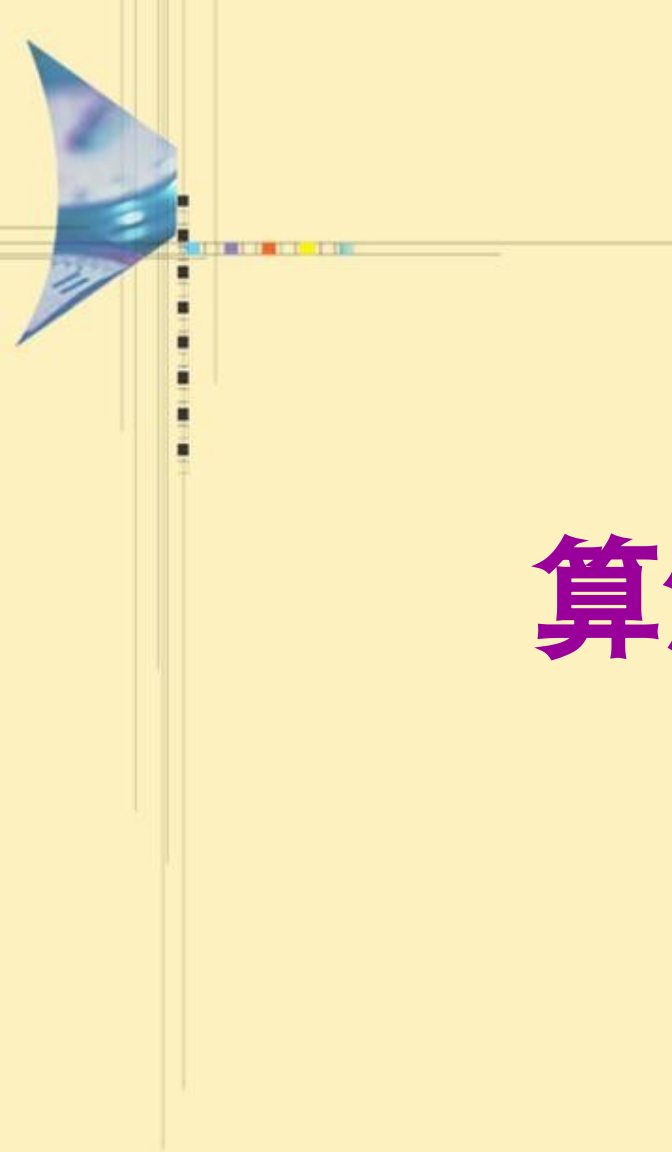
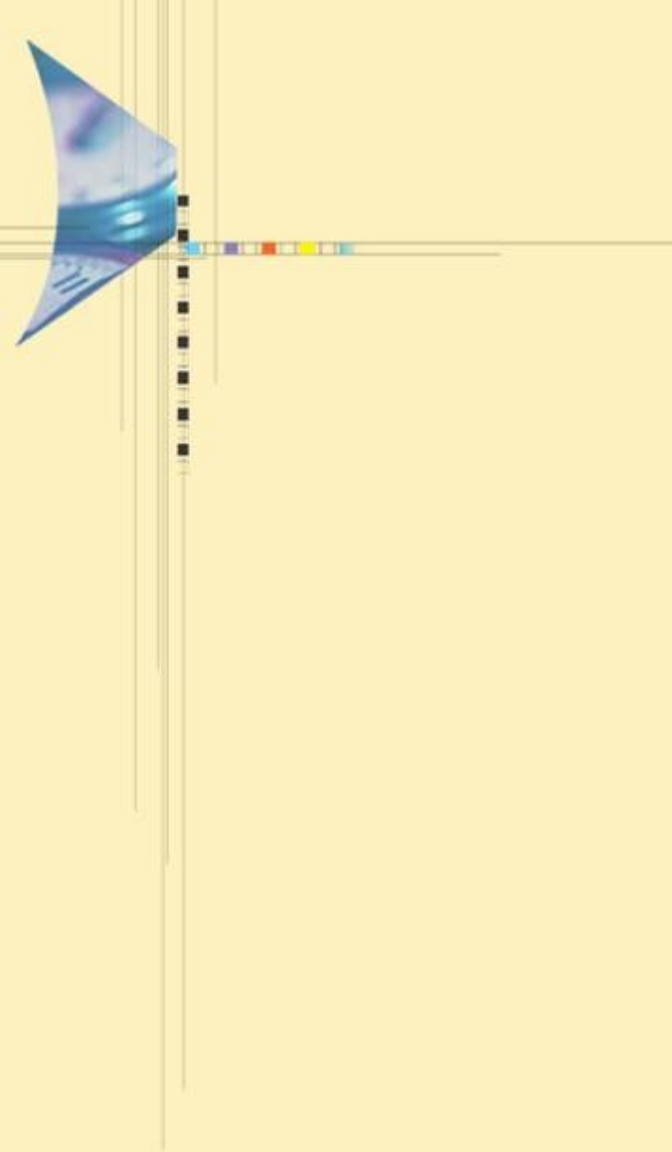




算法设计与分析





第一章

算法绪论

主要内容



1.1 算法的基本概念

1.2 算法分析基础

1.3 算法分析常用的数学知识

1.4 数据结构的表示、操作算法及分析



主要内容



1.1 算法的基本概念

1.1.1 算法的一般描述

1.1.2 算法的基本特性

1.1.3 算法与计算机程序的关系

1.1.4 算法与过程的关系

1.1.5 为什么要进行算法设计

1.1.6 算法与数据结构的关系



1.1.1 算法的一般描述

- 算法是在**有限**的步骤内求解某一问题，所使用的一组定义**明确**的规则。也就是在通过一组**有限定义**的指令来完成一些任务，给定一个**初始状态**，并相应的获取**最终结果**
- 算法通常由计算机程序来实现，算法设计的错误可能导致程序运行时的失败。

1.1.1 算法的一般描述

维基百科定义：

- An algorithm is an **effective method** that can be expressed within a finite amount of space and time[1] and in a well-defined formal language[2] for calculating a function.[3] Starting from an initial state and initial input (perhaps empty),[4] the instructions describe a computation that, when executed, proceeds through a finite[5] number of well-defined successive states, eventually producing "output"[6] and terminating at a final ending state.



1.1.2 算法的基本特性

1. 可行性

算法是**有效的**，充分的基础，原则上准确运行，人们用笔和纸在有限次运算内能完成。

2. 明确性

一个算法的每个步骤，必须精确地定义，每一种情况下，必须明确地指定要执行的每一步操作。

3. 有限性

一个算法必须保证有限数量的步骤后完成。

4. 输入

有零个或多个输入。以刻画运算的初始情况，0个输入是指算法本身定义了初始条件。

5. 输出

有一个或多个输出。以反映对数据加工后的结果，没有输出的算法是毫无意义的。

1.1.3 算法与计算机程序的关系

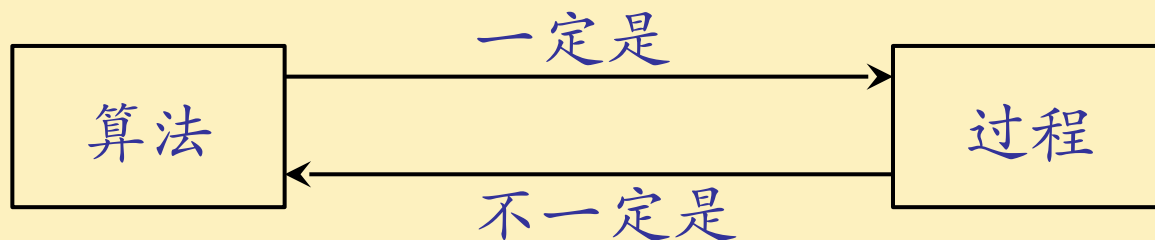
- 算法必须满足所有的五个特征。
- 程序只满足四个特征（可行性，明确性，输入，输出），不是一个规则的算法。
- 算法和程序的最大区别，算法是有限的，但是程序是可能无限的
- 例如：操作系统只要不关机就是一个无限期执行的程序。

1.1.4 算法与过程的关系

<< 算法导论 >> 第一部分第一节：

所谓**算法** (algorithm) 就是定义良好的**计算过程**。它取一个或者一组值作为输入，并产生一个或一组值作为输出。

亦即，**算法就是一系列的计算步骤**，用来将输入数据转换成输出结果。



1.1.5 为什么要进行算法设计

- 寻求解决一类问题的**方法**，使它可以通过计算机来完成。
- 把过程分解成若干个明确的**步骤**，然后用计算机能够接受的“语言”准确地描述出来，从而使计算机能够执行。

即：提出解决问题的方法和步骤



1.1.6 算法与数据结构的关系

- 程序 = 数据结构 + 算法。
- 算法和数据结构是相辅相成的关系。
- 数据结构是相互之间存在某种关系的数据元素的集合；算法是解决特定问题的有限求解步骤。
- 数据结构是算法实现的基础，设计一种算法时会构建适合于这种算法的数据结构；同时算法总是要依赖于某种数据结构实现。

主要内容



1.1 算法的基本概念

1.2 算法分析基础

1.3 算法分析常用的数学知识

1.4 数据结构的表示、操作算法及分析



主要内容



1.2 算法分析基础

1.2.1 算法分析的目的

1.2.2 算法分析的基本方法

1.2.3 时间复杂度

1.2.4 空间复杂度

1.2.5 什么是最优算法

1.2.6 算法正确性证明方法



1.2.1 算法分析的目的

- 算法分析是对一个算法需要多少计算时间和存储空间作定量的分析。一般计算出相应的数量级，常用时间复杂度和空间复杂度表示。
- 目的是评价算法的效率以求改进。要降低算法的时间复杂度和空间复杂度，提高算法的执行效率。

1.2.2 算法分析的基本方法

- **事前分析**：就算法本身，通过对其执行性能的理论分析，得出关于算法特性——时间和空间的一个特征函数（ O 、 Ω ）——与计算机软硬件没有直接关系。
- **事后测试**：将算法编制成程序后放到计算机上运行，收集其执行时间和空间占用等统计资料，进行分析判断——直接与物理实现有关。

1.2.3 时间复杂度

- 实际上，我们不需要精确的时间评估算法的效率，我们感兴趣的是输入数据量很大情况下的运行时间，因此，通常我们使用运行时间的增长速率和或增长趋势来评估效率。



例： $f(n)=n^2\log n+10n^2+n$ ，运行时间 $f(n)$ 是一个基于 $n^2\log n$ 增长的趋势。

- 当我们设置了一个包含低阶项和常量的函数，用它来表示一个算法的运行时间，此时可以说，我们正在测量算法的渐近运行时间，也就是渐进时间复杂度。



1.2.3 时间复杂度——基本操作

- 例子：

算术运算： $+$, $-$, $*$, $/$

比较和逻辑运算： $=$, $\&$, $|$, $\sim$

分配空间：`new`

- 基本操作由计算机在一**固定的执行时间**（不超过某一上界）内完成，基本操作与输入数据的长度或者使用何种算法无关。

1.2.3 时间复杂度——频度

- 频度定义：算法中语句或运算的执行次数。

- 例子：

$x=x+y$

(a)

for ($i=1;i \leq n;i++$)

$x=x+y;$

(b)

for ($i=1;i \leq n;i++$)

for ($j=1;j \leq n;j++$)

$x=x+y;$

(c)

- 分析：

(a) : $x=x+y$ 执行了 1 次

(b) : $x=x+y$ 执行了 n 次

(c) : $x=x+y$ 执行了 n^2 次

1.2.3 时间复杂度——构成

- 一个程序所占时间 $T(P) = \text{编译时间} + \text{运行时间}$;
- 编译时间与实例特征无关, 而且, 一般情况下, 一个编译过的程序可以运行若干次, 所以, 人们主要关注的是运行时间; 记作 t_p (实例特征)
- 如果了解所用编译器的特征, 就可以确定代码 P 进行加、减、乘、除、比较、读、写等基本操作所需的时间, 从而得到计算 t_p 的公式。令 n 代表实例特征 (这里主要是问题的规模), 则有如下的计算公式:

1.2.3 时间复杂度——构成

- $t_p(n) = caADD(n) + csSUB(n) + cmMUL(n) + cdDIV(n) + ccCMP(n) + \dots$
- 其中， ca ， cs ， cm ， cd ， cc 分别表示一次加、减、乘、除及比较操作所需的时间，函数 ADD ， SUB ， MUL ， DIV ， CMP 分别表示代码 P 中所使用的加、减、乘、除及比较操作的次数；

1.2.3 时间复杂度——构成

- 估算运行时间的方法：
 - A. 找一个或多个**关键操作**，确定这些关键操作所需的执行时间；
 - B. 确定程序总的**执行步数**。
- **关键操作**对时间复杂性影响最大。

1.2.3 时间复杂度——例 1 : 寻找最大元素

- ```
int Max(int *a,int n)
{ // 寻找 a[0:n-1] 中的最大元素
 int pos=0;
 for(i=1; i<n; i++)
 if(a[pos]<a[i])
 pos=i;
 return pos;
}
```

## 1.2.3 时间复杂度——例 1：寻找最大元素

- 这里关键操作是比较。for 循环中共进行了  $n-1$  次比较。for 循环每次执行之前都要比较一下  $i$  和  $n$  此外，Max 还进行了其它的操作，如初始化 pos、循环控制变量  $i$  的增量。这些一般都不包含在估算中，若纳入计数，则操作计数将增加一个常量。



## 1.2.3 时间复杂度——

### 例 2 : 计算 $a[0:n-1]$ 中元素的排名

```
template<class T>
void Rank(T a[],int n,int r[]){
 // 计算 $a[0:n-1]$ 中元素的排名
 for(int i=1; i<n; i++)
 r[i]=0; // 初始化
 // 逐对比较所有的元素
 for(int i=1;i<n;i++)
 for (j=0;j<i;j++)
 if (a[j]<a[i]) r[i]++;
 else r[j]++ ;
}
```

## 1.2.3 时间复杂度——

### 例 2 : 计算 $a[0:n-1]$ 中元素的排名

- 例中，对每个  $i$  的取值，执行比较的次数是  $i$ ，总的比较次数为  $1+2+\dots+(n-1)=(n-1)n/2$ 。在此，for 循环的额外开销、初始化数组  $r$  的开销、以及每次比较  $a$  中的两个数时对  $r$  进行的增值开销都未列入其中。

## 1.2.3 时间复杂度——

### 例 3 : 对数组 $a[0:n-1]$ 进行选择排序

```
template<class T>
void SelectionSort(T a[],int n){
 // 对数组 a[0:n-1] 中元素排序
 for(int size=n;size>1;size--){
 j=Max(a, size);
 Swap(a[j],a[size-1]);
 }
}
// 其中函数 Max 寻找最大元素定义如例 1
// 函数 Swap 由下述给出
template<class T>
inline void Swap(T &a, T &b){
 T temp=a; a=b; b=temp;
}
```

## 1.2.3 时间复杂度——

### 例 3 : 对数组 $a[0:n-1]$ 进行选择排序

- 例中给出的排序中, 首先找出最大元素, 把它移动到  $a[n-1]$ , 然后在余下的  $n-1$  个中再寻找最大的元素, 并把它移动到  $a[n-2]$ , 如此进行下去。从例 1 中已经知道, 每次调用  $\text{Max}(a, \text{size})$  需要进行  $\text{size}-1$  次比较, 所以总的比较次数为  $1+2+\dots+n-1=(n-1)n/2$ . 每调用一次函数  $\text{Swap}$  需要执行三次元素移动, 所以总的移动次数为  $3(n-1)$ . 当然, 在本程序中, 还会有其他开销, 在此未予考虑。总的比较次数为  $(n-1)n/2+3(n-1)$ 。

## 1.2.3 时间复杂度—— 最好、最坏、平均情况分析

- 实际算法分析过程中，人们往往还会关心最好的、最坏的和平均的操作步数是多少。在上面的计算中我们都考虑的是最坏的情况。

## 1.2.3 时间复杂度—— 最好、最坏、平均情况分析

- 最坏情况下的时间复杂性：

$$T_{\max}(N) = \max_{I \in D_N} T(N, I) = T(N, I^*)$$

- 最好情况下的时间复杂性：

$$T_{\min}(N) = \min_{I \in D_N} T(N, I) = T(N, \tilde{I})$$

- 平均情况下的时间复杂性：

- $$T_{\text{avg}}(N) = \sum_{I \in D_N} P(I) T(N, I)$$

- 其中  $D_N$  是规模为  $N$  的合法输入的集合； $I^*$  是  $D_N$  中使  $T(N, I^*)$  达到  $T_{\max}(N)$  的合法输入； $\tilde{I}$  是中使  $T(\tilde{I}, N)$  达到  $T_{\min}(N)$  的合法输入；而  $P(I)$  是在算法的应用中出现输入  $I$  的概率。

## 1.2.3 时间复杂度——

### 例 4 : 顺序查找算法

```
template<class Type>
int seqSearch(Type *a, int n, Type k){
 for(int i=0;i<n;i++)
 if (a[i]==k) return i;
 return -1;
}
```



## 1.2.3 时间复杂度——

### 例 4：顺序查找算法

- $T_{\max}(n) = \max \{ T(I) \mid \text{size}(I)=n \} = O(n)$
- $T_{\min}(n) = \min \{ T(I) \mid \text{size}(I)=n \} = O(1)$
- 在平均情况下，假设：
  - 搜索成功的概率为  $p$  ( $0 \leq p \leq 1$ )；
  - 在数组的每个位置  $i$  ( $0 \leq i < n$ ) 搜索成功的概率相同，均为  $p/n$ 。

$$T_{\text{avg}}(n) = \sum_{\text{size}(I)=n} p(I)T(I) = \frac{p}{n} \sum_{i=1}^n i + n(1-p) = \frac{p(n+1)}{2} + n(1-p)$$

$$= \left( 1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + 3 \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n} \right) + n \cdot (1-p)$$

- 当  $p=1$ ，则  $T_{\text{avg}}(n) = (n+1)/2$ ；当  $p=0$ ，则  $T_{\text{avg}}(n) = n$ ；

## 1.2.3 时间复杂度——渐进表示

记：算法的计算时间为  $f(n)$

数量级限界函数为  $g(n)$

其中，

- $n$  是输入或输出规模的某种测度，即问题的规模。
- $f(n)$  表示算法的“实际”执行时间。
- $g(n)$  是形式简单的函数，如  $n^m$ ， $\log n$ ， $2n$ ， $n!$  等。  
是事前分析中通过对计算时间或频率计数统计分析所得的与机器及语言无关的函数。

# 渐进表示——大 O- 记号 ( 渐近上界记号 )

## 定义 1.2

- 设  $f(n)$  和  $g(n)$  是定义在正数集上的正函数。
- 如果存在正的常数  $c$  和自然数  $n_0$  , 使得当  $n \geq n_0$  时有  $f(n) \leq c g(n)$  , 则称函数  $f(n)$  当  $n$  充分大时上有界, 且  $g(n)$  是它的一个上界, 记为  $f(n)=O(g(n))$  。即  $f(n)$  的阶不高于  $g(n)$  的阶。

也就是说, 当  $\lim_{n \rightarrow \infty} f(n) / g(n)$  存在, 且

$$\lim_{n \rightarrow \infty} f(n) / g(n) \neq \infty \quad \text{则表示 } f(n)=O(g(n))$$

- 根据定义可知: 函数  $f(n)$  至多是函数  $g(n)$  的  $c$  倍, 除非  $n < n_0$  。即是说, 当  $n$  充分大时,  $g(n)$  是  $f(n)$  的一个上界函数, 在相差一个非零常数倍的情况下。

# 渐进表示——大 O- 记号 ( 渐近上界记号 )

- 例：

$f(n) = 10n^2 + 20n$ . Then,

$f(n) = O(n^2)$  由于当  $n \geq 1$ ,  $f(n) \leq 30n^2$ .

Consequently,  $\lim_{n \rightarrow \infty} f(n)/n^2 = 10 \neq \infty$ .

- $C_0 = O(1)$ :  $f(n)$  等于非零常数的情形。
- $3n + 2 = O(n)$ : 可取  $c = 4$ ,  $n_0 = 2$ 。
- $100n + 6 = O(n)$ : 可取  $c = 101$ ,  $n_0 = 6$ 。
- $10n^2 + 4n + 3 = O(n^2)$ : 可取  $c = 11$ ,  $n_0 = 6$ 。
- $6 \times 2^n + n^2 = O(2^n)$ : 可取  $c = 7$ ,  $n_0 = 4$ 。

# 渐进表示——大 O- 记号 ( 渐近上界记号 )

• 例：

• 因为  $\lim_{n \rightarrow \infty} \frac{3n+2}{n} = 3$ , 所以  $3n+2=O(n)$ ;

• 因为  $\lim_{n \rightarrow \infty} \frac{10n^2+4n+2}{n^2} = 10$ , 所以  $10n^2+4n+2=O(n^2)$ ;

• 因为  $\lim_{n \rightarrow \infty} \frac{6*2^n+n^2}{2^n} = 6$ , 所以  $6*2^n+n^2=O(2^n)$ ;

• 因为  $\lim_{n \rightarrow \infty} \frac{n^{16}+3*n^2}{2^n} = 0$ , 所以  $n^{16}+3*n^2=O(2^n)$ ;

$$n^3 + n^2 \log n = O(n^3);$$

$$n^4 + n^{2.5} \log^{20} n = O(n^4); 2^n n^4 \log^3 n + 2^n n^5 / \log^3 n = O(2^n n^5).$$

# 渐进表示——大 O- 记号 ( 渐近上界记号 )

- 注意事项
- 1. 用来作比较的函数  $g(n)$  应该尽量接近所考虑的函数  $f(n)$ .  
 $3n+2=O(n^2)$  松散的界限;  $3n+2=O(n)$  较好的界限。
- 2.  $f(n)=O(g(n))$  不能写成  $g(n)=O(f(n))$  , 因为两者并不等价。  
实际上, 这里的等号并不是通常相等的含义。按照定义, 用集合符号更准确些。
- 3.  $O(g(n))=\{f(n)|f(n) \text{ 满足: 存在正的常数 } c \text{ 和 } n_0, \text{ 使得当 } n \geq n_0 \text{ 时, } f(n) \leq cg(n)\}$ 。所以, 人们常常把  $f(n)=O(g(n))$  读作: “ $f(n)$  是  $g(n)$  的一个大 O 成员”。



# 渐进表示—— $\Omega$ - 记号 ( 渐近下界记号 )

## 定义 1.3

- 如果存在正的常数  $c$  和自然数  $n_0$  , 使得当  $n \geq n_0$  时有  $f(n) \geq cg(n)$  , 则称函数  $f(n)$  当  $n$  充分大时下有界, 且  $g(n)$  是它的一个下界, 记为  $f(n) = \Omega(g(n))$  。即  $f(n)$  的阶不低于  $g(n)$  的阶

也就是说, 当  $\lim_{n \rightarrow \infty} f(n) / g(n)$  存在, 且

$$\lim_{n \rightarrow \infty} f(n) / g(n) \neq 0 \quad \text{则表示} \quad f(n) = \Omega(g(n))$$

- 根据定义可知: 函数  $f$  至少是函数  $g$  的  $c$  倍, 除非  $n < n_0$  。  
即是说, 当  $n$  充分大时,  $g$  是  $f$  的一个下界函数, 在相差一个非零常数倍的情况下。

- 例:

Let  $f(n) = 10n^2 + 20n$ . Then,

$f(n) = \Omega(n^2)$  由于当  $n \geq 1$ ,  $f(n) \geq n^2$ .

Consequently,  $\lim_{n \rightarrow \infty} f(n)/n^2 = 10 \neq 0$ .



# 渐进表示—— $\Theta$ - 记号 ( 紧渐进界记号 )

## 定义 1.4

- 如果存在 2 个正的常数  $c_1, c_2$  和自然数  $n_0$  , 使得当  $n \geq n_0$  时有  $c_1 g(n) \leq f(n) \leq c_2 g(n)$  ,

也就是说, 当  $\lim_{n \rightarrow \infty} f(n) / g(n)$  存在, 并且  $\lim_{n \rightarrow \infty} f(n) / g(n) = c$  , 则  $f(n) = \Theta(g(n))$ ,  $c$  是大于 0 的常量.

可以推断, 当且仅当  $f(N) = O(g(N))$  且  $f(N) = \Omega(g(N))$  。此时称  $f(N)$  与  $g(N)$  同阶。

- 根据定义可知: 函数  $f$  介于函数  $g$  的  $c_1$  和  $c_2$  倍之间, 即当  $n$  充分大时,  $g$  既是  $f$  的下界, 又是  $f$  的上界.

# 渐进表示—— $\Theta$ - 记号 ( 紧渐进界记号 )

• 例：

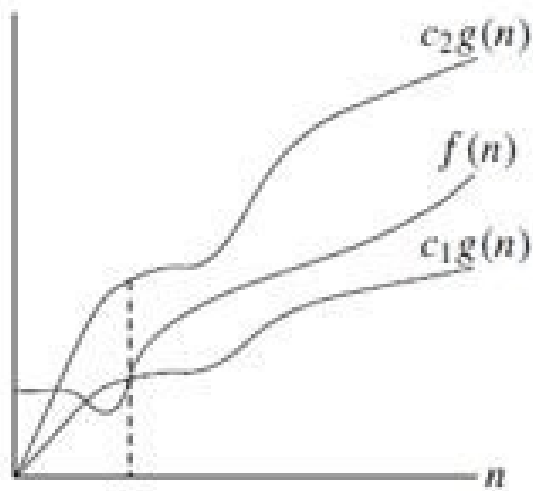
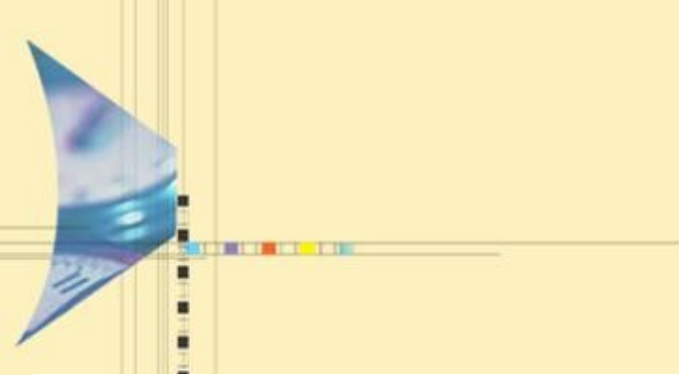
Let  $f(n) = 10n^2 + 20n$ . Then,

$f(n) = \Theta(n^2)$  由于当  $n \geq 1$ ,  $n^2 \leq f(n) \leq 30n^2$ .

Consequently,  $\lim_{n \rightarrow \infty} f(n)/n^2 = 10$ .

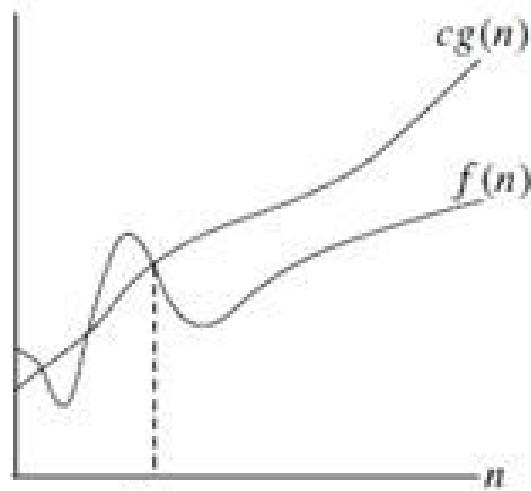
- $3n + 2 = \Theta(n)$ ,
- $10n^2 + 4n + 2 = \Theta(n^2)$  。
- $5 \times 2^n + n^2 = \Theta(2^n)$





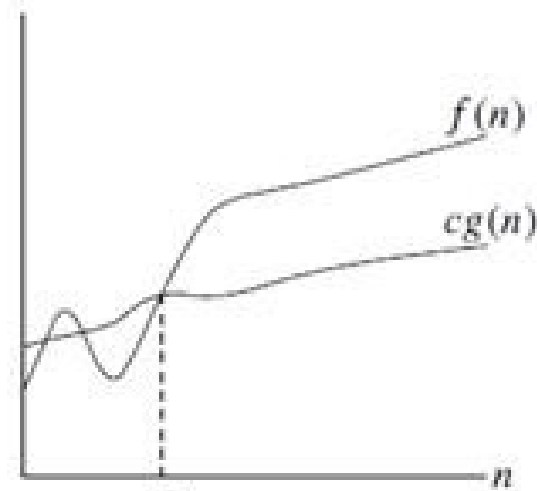
$$f(n) = \Theta(g(n))$$

(a)



$$f(n) = O(g(n))$$

(b)



$$f(n) = \Omega(g(n))$$

(c)

# 渐进表示——□ - 记号 ( 例：折半查找 )

```
• Template <class T>
• Int BinarySearch(T a[], const T & x, int n)
• { // 在 a[0]<=a[1]<=...<=a[n-1] 中搜索 x
• // 如果找到，则返回所在位置，否则返回 -1
• int left=0; int right=n-1;
• while(left<=right) {
• int middle=(left+right)/2;
• if (x==a[middle]) return middle;
• if (x>a[middle]) left=middle+1;
• else right=middle-1;
• }
• Return -1 ; // 未找到 x
• }
```

# 渐进表示——□ - 记号（例：折半查找）

- while 的每次循环（最后一次除外）都将以减半的比例缩小搜索范围，所以，该循环在最坏的情况下需要执行  $\Theta(\log n)$  次。由于每次循环需耗时  $\Theta(1)$ ，因此在最坏情况下，总的时间复杂性为  $\Theta(\log n)$ 。

# 渐进表示——小 $o$ - 记号 ( 非紧上界记号 )

## 定义 1.5

- 对于任意给定的  $c > 0$  , 都存在正整数  $n_0$  , 使得当  $n > n_0$  时有  $f(n) < cg(n)$  , 则称函数  $f(n)$  当  $n$  充分大时的阶比  $g(n)$  低, 记为  $f(n) = o(g(n))$  。
  - 也就是说, 当  $\lim_{n \rightarrow \infty} f(n) / g(n)$  存在, 并且
$$\lim_{n \rightarrow \infty} f(n) / g(n) = 0$$
 则  $f(n) = o(g(n))$ .
- 可以推断:  $f(n) = o(g(n))$  成立前提: 必须满足  $f(n) = O(g(n))$  , 同时满足  $g(n) \neq O(f(n))$ .

# 渐进表示——小 $\Omega$ - 记号 (非紧下界记号)

## 定义 1.5

- 对于任意给定的  $c > 0$  , 都存在正整数  $n_0$  , 使得当  $n > n_0$  时有  $f(n) > cg(n)$  , 则称函数  $f(n)$  当  $n$  充分大时的阶比  $g(n)$  高, 记为  $f(n) = \omega(g(n))$  。

- 也就是说, 当  $\lim_{n \rightarrow \infty} f(n) / g(n)$  存在, 并且

$$\lim_{n \rightarrow \infty} f(n) / g(n) \rightarrow \infty \text{ 则 } f(n) = \omega(g(n))$$



# 渐近记号在等式和不等式中的意义

- $f(n) = \Theta(g(n))$  的确切意义是:  $f(n) \in \Theta(g(n))$ 。
- 一般情况下, 等式和不等式中的渐近记号  $\square(g(n))$  表示  $\square(g(n))$  中的某个函数。  
例如:  
 $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$  表示  
 $2n^2 + 3n + 1 = 2n^2 + f(n)$ , 其中  $f(n)$  是  $\square(n)$  中某个函数。
- 等式和不等式中渐近记号  $O$ ,  $\Omega$  和  $\square$  的意义是类似的。

# 渐近记号的若干性质

- (1) 传递性:
- $f(n) = \Theta(g(n))$  ,  $g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$  ;
- $f(n) = O(g(n))$  ,  $g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$  ;
- $f(n) = \Omega(g(n))$  ,  $g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n))$  ;
- $f(n) = o(g(n))$  ,  $g(n) = o(h(n)) \Rightarrow f(n) = o(h(n))$  ;
- $f(n) = \omega(g(n))$  ,  $g(n) = \omega(h(n)) \Rightarrow f(n) = \omega(h(n))$  ;

# 渐近记号的若干性质

- (2) 反身性:

- $f(n) = \Theta(f(n))$  ;

- $f(n) = O(f(n))$  ;

- $f(n) = \Omega(f(n))$ .

- (3) 对称性:

- $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$  .

- (4) 互对称性:

- $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$  ;

- $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$  ;

# 渐近记号的若干性质

- (5) 算术运算:
- $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$  ;
- $O(f(n)) + O(g(n)) = O(f(n) + g(n))$  ;
- $O(f(n)) * O(g(n)) = O(f(n) * g(n))$  ;
- $O(cf(n)) = O(f(n))$  ;
- $g(n) = O(f(n)) \Rightarrow O(f(n)) + O(g(n)) = O(f(n))$  。

# 常用阶的比较

| $n$  | 1 | $\lg n$ | $n$  | $n \lg n$ | $n^2$     | $n^3$     | $2^n$                  |
|------|---|---------|------|-----------|-----------|-----------|------------------------|
| 1    | 1 | 0.00    | 1    | 0         | 1         | 1         | 2                      |
| 10   | 1 | 3.32    | 10   | 33        | 100       | 1000      | 1024                   |
| 100  | 1 | 6.64    | 100  | 664       | 10,000    | 1,000,000 | $1.27 \times 10^{30}$  |
| 1000 | 1 | 9.97    | 1000 | 9970      | 1,000,000 | $10^9$    | $1.07 \times 10^{301}$ |

## 1.2.4 空间复杂度

为什么要考虑空间复杂性：

- 在多用户系统中运行时，需指明分配给该程序的内存大小；
- 想预先知道计算机系统是否有足够的内存来运行该程序；
- 一个问题可能有若干个不同的内存需求解决方案，从中择取；
- 用空间复杂性来估计一个程序可能解决的问题的最大规模。

## 1.2.4 空间复杂度

- 定义：空间复杂度 (Space Complexity) 是对一个算法在运行过程中临时占用存储空间大小的量度，记做  $S(n)=O(f(n))$ 。其中  $n$  为问题的规模 (或大小)。
- 一个算法在计算机存储器上所占用的存储空间，包括存储算法本身所占用的存储空间，算法的输入输出数据所占用的存储空间和算法在运行过程中临时占用的存储空间这三个方面。



## 1.2.4 空间复杂度——程序所需要的空间

- 在分析空间复杂性中，**实例特征**的概念特别重要。
- 所谓**实例特征**是指决定问题规模的那些因素。如，输入和输出的数量或相关数的大小，如对  $n$  个元素进行排序、 $n \times n$  矩阵的加法等。都可以  $n$  作为实例特征，两个  $m \times n$  矩阵的加法应该以  $n$  和  $m$  两个数作为实例特征。
- 一个程序所需要的空间可分为两部分：
  - **固定部分**，它独立于实例特征。主要包括指令空间、简单变量以及定义复合变量所占用的空间、常量所占用的空间。
  - **可变部分**，主要包括复合变量所需的空间（其大小依赖于所解决的具体问题）。动态分配的空间（依赖于实例特征）、递归栈所需的空间（依赖于实例特征）。

## 1.2.4 空间复杂度——程序所需要的空间

- 令  $S(P)$  表示程序  $P$  所需的空间，则有

$$S(P) = c + S_p(\text{实例特征})$$

- 其中：
  - a)  $c$  表示固定部分所需要的空间，是一个常数；
  - b)  $S_p$  表示可变部分所需的空间。在分析程序的空间复杂性时，一般着重于估算  $S_p(\text{实例特征})$ 。

## 1.2.4 空间复杂度——程序所需要的空间

1. **指令空间** --- 用来存储经过编译之后的程序空间。

程序所需的指令空间的大小取决于如下因素：

- 把程序编译成机器代码的编译器；
- 编译时实际采用的编译器选项；
- 目标计算机。



## 1.2.4 空间复杂度——程序所需要的空间

2. 数据空间 --- 用来存储所有常量和变量的值。分成两部分：

- a.) 存储常量和简单变量；
  - b.) 存储复合变量。
- 前者所需的空間取决于所使用的计算机和编译器，以及变量与常量的数目，这是由于我们往往是指计算所需内存的字节数，而每个字节所占的数位依赖于具体的机器环境。
  - 后者包括数据结构所需的空間及动态分配的空間。

## 1.2.4 空间复杂度——字节数

| 类型            | 占用字节数 | 适用范围               |
|---------------|-------|--------------------|
| char          | 1     | -128-127           |
| unsigned char | 1     | 0 – 255            |
| short         | 2     | -32768-32767       |
| int           | 2     | -32768-32767       |
| unsigned int  | 2     | 0-65535            |
| long          | 4     | -231- 231-1        |
| unsigned long | 4     | 0- 231-1           |
| float         | 4     | $\pm 3.4E \pm 38$  |
| double        | 8     | $\pm 1.7E \pm 308$ |
| pointer       | 2     |                    |

## 1.2.4 空间复杂度——程序所需要的空间

3. 环境栈空间 --- 保存函数调用返回时恢复运行所需要的

信息。当一个函数被调用时，下面数据将被保存在环

境栈中：

- 返回地址；
- 所有局部变量的值、递归函数的传值形式参数的值；
- 所有引用参数以及常量引用参数的定义。

## 1.2.4 空间复杂度——例 1 利用应用参数

- Template <class T>
- T abc( T &a, T &b, T &c)
- {
- Return  $a+b+c+b*c+(a+b+c)/(a+b)+4$ ;
- }
- 在例 1 中，采用 T 作为实例特征。由于 a,b,c 都是引用参数，在函数中不需要为它们的值分配空间，但需保存指向这些参数的指针。若每个指针需要 2 个字节，则共需要 6 个字节的指针空间，此时函数所需要的总空间是一个常量，而 Sabc(实例特征)=0。
- 但若函数 abc 的参数是传值参数，则每个参数需要分配空间 sizeof(T) 的空间，于是 a,b,c 所需的空间为： $3 \times \text{sizeof}(T)$ 。函数 abc 所需要的其他空间都与 T 无关，故 Sabc(实例特征)= $3 \times \text{sizeof}(T)$ 。



## 1.2.4 空间复杂度——例 2 顺序查找

在  $a[0:n-1]$  中搜索  $x$ ，若找到则返回所在的位置，否则返回 -1

```
template <class T>
int SequentialSearch(T a[],const T &x, int n)
{
 int i;
 for(i=0; i<n && a[i]!=x; i++);
 if (i==n) return -1;
 return i;
}
```

## 1.2.4 空间复杂度——例 2 顺序查找

- 在例 2 中，假定采用被查数组的长度  $n$  作为实例特征，并取  $T$  为 `int` 类型。
- 数组中每个元素需要 2 个字节，实参  $x$  需要 2 个字节，传值形式参数  $n$  需要 2 个字节，局部变量  $i$  需要 2 个字节，整数常量  $-1$  需要 2 个字节，所需要的总的空间为 10 个字节，其独立于  $n$ ，所以  $S(n)=0$ 。
- 这里我们并没有把数组  $a$  所需的空间计算进来，因为该数组所需的空间已在定义实际参数（对应于  $a$ ）的函数中分配，不能再到函数 `SequentialSearch` 所需的空间上去。

## 1.2.5 最优算法

- 定义如果可以证明任何一个求解问题 $\square$ 的算法必定是 $\square(f(n))$ ，那么我们把在 $O(f(n))$ 时间内求解问题 $\square$ 的任何算法都称为问题 $\square$ 的**最优算法**。
- 文献中被广泛使用的定义没有考虑空间复杂性，原因如下：
  - 1) 时间比空间珍贵；
  - 2) 大多数最优算法的空间复杂性同处 $O(n)$ 阶内。

## 1.2.6 算法正确性证明方法

### 概述：

- 对于明显是正确的算法（或程序）可以通过上机调试、验证其正确性。调试用的数据要精心挑选，以期达到算法对“所有的”数据都是正确的目标。
- 只要找出一组数据使算法失败，就能否定整个算法的正确性；
- 算法的正确性一般通过数学方法进行推理证明，常用的数学方法除了直接证明法外，还有反证法和数学归纳法。

## 1.2.6 算法正确性证明——反证法

- ①要证明定理  $T$  是正确的，首先假定  $T$  是错误的。
- ②用正确的命题和推理规则进行推理，若出现下列条件之一，就会得出矛盾的结果：
  - 与已知条件矛盾
  - 与公理矛盾
  - 与证明过的定理矛盾
  - 自相矛盾
- ③由此推出定理  $T$  是正确的。

## 1.2.6 算法正确性证明——反证法

例子：证明没有最大的整数。

证明：用反证法。

①反面假设：

假设存在一个最大整数，记为  $N$ 。

②由假设推出矛盾：

设  $P=N+1$ ，因为  $P$  是两个整数的和，所以  $P$  也是整数，

且  $P>N$ ，与  $N$  是最大整数矛盾。

③肯定原来的结论：

故没有最大的整数。证毕

## 1.2.6 算法正确性证明——数学归纳法

- 数学归纳法，通常被用于证明某个给定命题（定理）在整个（或者局部）自然数范围内成立。
- 数学归纳法依靠假设的事实来证明定理，是用“有限”步骤解决“无限”步骤问题的一种严格证明方法。



## 1.2.6 算法正确性证明——数学归纳法

例子：金字塔求和。

证明：用数学归纳法。

|                        |   |     |
|------------------------|---|-----|
| 1                      | = | 1   |
| 3 + 5                  | = | 8   |
| 7 + 9 + 11             | = | 27  |
| 13 + 15 + 17 + 19      | = | 64  |
| 21 + 23 + 25 + 27 + 29 | = | 125 |

当  $i=1$  时， $i^3 = 1$ , 假设成立；

当  $i=k$  时，若假设成立，则

两行之差： $(k+1)^3 - k^3$ ，

两行第一个数之差： $2 * k$

两行最后的数之差： $2 * (k$

则  $k+1$  行末数： $(k+1)^3 - k^3 -$

则两行最后数之差： $k^2 + 3k$

证毕，假设成立

# 主要内容



1.1 算法的基本概念

1.2 算法分析基础

1.3 算法分析常用的数学知识

1.4 数据结构的表示、操作算法及分析



# 主要内容



## 1.3 算法分析常用的数学知识

1.3.1 集合、关系、函数

1.3.2 证明方法

1.3.3 对数

1.3.4 底函数、顶函数

1.3.5 阶乘和二项式系数

1.3.6 鸽巢原理

1.3.7 和式

1.3.8 递推关系

## 1.3.1 集合、关系、函数

- 当分析算法时，我们认为它的输入是从某个特定范围（如整数集合）取出的一个集合。形式的说，可以认为算法是一个函数，它是一个受约束的关系，它映射每一个可能的输入到一个特定的输出。
- 这样，集合和函数都处于算法分析的核心中

## 1.3.1 集合、关系、函数

- 集合
  - “集合”可用来指任何一批对象，它们成为集合的成员或元素。
- 有限 / 无限集合
  - 如果对于某个常数  $n_0$ ，一个集合包含  $n$  个元素，则称它为有限的，否则是无限的。
- 可数 / 不可数集合
  - 如果一个无限集合的元素能被第一个元素、第二个元素列举出来，就称这个无限集合是可数的，否则称为不可数的。
  - 整数集合可数，实数集合不可数。
- 集合的描述
  - 有限整数集合  $\{1,2,3\}$ ;
  - 自然数集合  $\{1,2,3,\dots\}$ ;
  - 有限整数集合  $\{x|1\leq x\leq 100 \text{ and } x \text{ 是整数}\}$ ,
  - 空集  $\{\}$  or  $\emptyset$ .

## 1.3.1 集合、关系、函数

- 基数
  - 如果  $A$  是一个有限集，那么  $A$  的基数是指  $A$  中元素的个数，表示成  $|A|$ 。
- 成员（元素）
  - 如果  $x$  是  $A$  的成员，我们用  $x \in A$  表示，否则  $x \notin A$ 。
- 子集
  - 如果每一个  $B$  的元素都是  $A$  的元素，那么称集合  $B$  是集合  $A$  的子集合，记为  $B \subseteq A$ ，如果还有  $B \neq A$ ，则称  $B$  是  $A$  的真子集，记为  $B \subset A$ 。  
如：  $\{1, 3\} \subset \{1, 2, 3, 4\}$       $\{1, 2, 3, 4\} \subseteq \{1, 2, 3, 4\}$
- 幂集
  - 集合  $A$  的幂集是  $A$  的所有子集的集合，记为  $P(A)$ ，如果  $|A| = n$ ，那么  $|P(A)| = 2^n$ 。

## 1.3.1 集合、关系、函数

- 并集
  - 两个集合的并是集合  $\{x|x \in A \text{ 或 } x \in B\}$ ，记为  $A \cup B$ 。
- 交集
  - 两个集合的交是集合  $\{x|x \in A \text{ 且 } x \in B\}$ ，记为  $A \cap B$ 。两个不相交集  $A \cap B = \emptyset$ 。
- 差集
  - 集合  $A$  和集合  $B$  的差是集合  $\{x|x \in A \text{ 且 } x \notin B\}$ ，记为  $A - B$ 。
- 补集
  - 集合  $A$  的补集定义为  $U - A$ ，这里  $U$  是包含  $A$  在内的全集，它通常可以从上下文中知道， $A$  的补集可以记为  $\bar{A}$ 。



## 1.3.1 集合、关系、函数

- 有序  $n$  元组

- 有序  $n$  元组  $(a_1, a_2, \dots, a_n)$  是一个有序集，其中  $a_1$  是它的第一个元素， $a_2$  是它的第二个元素 $\dots$ ， $a_n$  是它的第  $n$  个元素。作为特例，二元组称为有序对。

- 笛卡尔积

- 令  $A$  和  $B$  是两个集合， $A$  和  $B$  的笛卡尔积是所有有序对  $(a, b)$  的集合，这里  $a \in A$ ， $b \in B$ ，笛卡尔积记为  $A \times B$ 。用集合符号表示：

$$A \times B = \{(a, b) \mid a \in A \text{ and } b \in B\}.$$

- 更一般的， $A_1, A_2, \dots, A_n$  的笛卡尔积定义为

$$A_1 \times A_2 \times \dots \times A_n = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i, 1 \leq i \leq n\}.$$

## 1.3.1 集合、关系、函数

### 笛卡尔积举例

- :  $A$  表示某大学所有学生的集合,  $B$  表示大学开设的所有课程的集合, 则  $A \times B$  可以用来表示该校学生选课的所有可能情况。令  $A$  是直角坐标系中  $x$  轴上的点集,  $B$  是直角坐标系中  $y$  轴上的点集, 于是  $A \times B$  就和平面点集一一对应。
- 设  $A = \{a, b\}$ ,  $B = \{0, 1, 2\}$ , 则
$$A \times B = \{ \langle a, 0 \rangle, \langle a, 1 \rangle, \langle a, 2 \rangle, \langle b, 0 \rangle, \langle b, 1 \rangle, \langle b, 2 \rangle \}$$
$$B \times A = \{ \langle 0, a \rangle, \langle 0, b \rangle, \langle 1, a \rangle, \langle 1, b \rangle, \langle 2, a \rangle, \langle 2, b \rangle \}$$
- 如果  $|A| = m$ ,  $|B| = n$ , 则  $|A \times B| = mn$ 。

## 1.3.1 集合、关系、函数

- 关系

- 设  $A$  和  $B$  是两个非空集，一个从  $A$  到  $B$  的 **二元关系**，或简单的说  **$A$  到  $B$  的关系  $R$**  是有序对  $(a,b)$  的集合，这里  $a \in A$ ， $b \in B$ ，就是  $R \subseteq A \times B$ 。如果  $A=B$ ，则称  **$R$  是集合  $A$  上的关系**。
- 如果一个集合满足以下条件之一：（1）集合非空，且它的元素都是有序对；（2）集合是空集；则称该集合为一个 **二元关系**，记作  $R$ 。二元关系也可简称为关系。对于二元关系  $R$ ，如果  **$\langle x,y \rangle \in R$** ，可记作  **$xRy$** 。
- 设  $R1 = \{\langle 1,2 \rangle, \langle a,b \rangle\}$ ， $R2 = \{\langle 1,2 \rangle, a, b\}$ 。则  $R1$  是二元关系， $R2$  不是二元关系，只是一个集合，除非将  $a$  和  $b$  定义为有序对。
- $R = \{\langle \text{上}, \text{下} \rangle, \langle \text{前}, \text{后} \rangle, \langle \text{正}, \text{反} \rangle, \langle \text{左}, \text{右} \rangle\}$  是否为二元关系？

## 1.3.1 集合、关系、函数

- R 的定义域

- R 的定义域，有时写成  $\text{Dom}(R)$ ，定义为

$$\text{Dom}(R) = \{a \mid \text{对某个 } b \in B, (a, b) \in R\}$$

- R 的值域

- R 的值域，有时写成  $\text{Ran}(R)$ ，定义为

$$\text{Ran}(R) = \{b \mid \text{对某个 } a \in A, (a, b) \in R\}$$

## 1.3.1 集合、关系、函数

- 例子

设  $R_1=\{(2,5), (3,3)\}$ ,

$R_2=\{(x,y)|x, y \text{ 是正整数, 且 } x \leq y\}$  ,

$R_3=\{(x,y)|x, y \text{ 是实数, 且 } x^2+y^2 \leq 1\}$ .

那么  $\text{Dom}(R_1)=\{2,3\}$  ,  $\text{Ran}(R_1)=\{5,3\}$ ,  $\text{Dom}(R_2)=\text{Ran}(R_2)$  是自然数集合,  $\text{Dom}(R_3)=\text{Ran}(R_3)$  是在区间  $[-1..1]$  中的实数集合.

## 1.3.1 集合、关系、函数

设  $R$  是在集合  $A$  上的关系。

- 自反 / 反自反

- 如果对所有的  $a \in A$  , 有  $(a,a) \in R$  , 则称  $R$  是自反的;
- 如果对所有的  $a \in A$  , 有  $(a,a) \notin R$  , 则称  $R$  是反自反的;

- 对称 / 非对称 / 反对称

- 如果  $(a,b) \in R$  蕴含着  $(b,a) \in R$  , 则称  $R$  是为对称的;
- 如果  $(a,b) \in R$  蕴含着  $(b,a) \notin R$  , 则称  $R$  是为非对称的;
- 如果  $(a,b) \in R$  ,  $(b,a) \in R$  , 蕴含着  $a=b$  , 则称  $R$  是为反对称的;

- 传递

- 如果  $(a,b) \in R$  并且  $(b,c) \in R$  得出  $(a,c) \in R$  , 则称  $R$  是传递的。

- 偏序

- 一个关系是自反的、反对称的和传递的, 则称为是一个偏序。

## 1.3.1 集合、关系、函数

- 例子
- 设：

$R1 = \{(x, y) | x, y \text{ 是正整数, 且 } x \text{ 整除 } y\}$   $y \text{ 能整除 } z, \text{ 则 } x \text{ 能整除 } z$  ;

$R2 = \{(x, y) | x, y \text{ 是整数, 且 } x \leq y\}$ .

那么  $R1$  和  $R2$  都是自反的、反对称的和传递的，因此都是偏序。

分析  $R1$  :

- 自反的：  $x$  可以整除  $x$  ;
- 反对称的：如果  $x$  能整除  $y$  ,  $y$  也能整除  $x$  , 则  $x=y$  ;
- 传递的：如果  $x$  能整除  $y$  ,  $y$  能整除  $z$  , 则  $x$  能整除  $z$  ;



## 1.3.1 集合、关系、函数

- 等价关系

- 如果集合  $A$  上的关系是自反的、对称的和传递的，则称它是等价关系。

- 等价类

- $R$  是  $A$  上的等价关系，此情况下， $R$  划分  $A$  成为等价类  $C_1, C_2, \dots, C_k$ ，这样等价类中的任意两个元素有  $R$  关系。也就是对于任意  $C_i, 1 \leq i \leq k$ ，如果  $x \in C_i$  且  $y \in C_i$ ，则  $(x, y) \in R$ 。另一方面，如果  $x \in C_i$  且  $y \in C_j$ ， $i \neq j$ ，那么  $(x, y) \notin R$ 。

## 1.3.1 集合、关系、函数

- 例子
- 设  $x$  和  $y$  是两个整数， $n$  是正整数，如果对于某个整数  $k$ ，有  $x-y=kn$ ，则称  $x$  和  $y$  模  $n$  同余，记为

$$x \equiv y \pmod{n}$$

换句话说，如果  $x$  和  $y$  被  $n$  除的时候，余数相同，就有  $x \equiv y \pmod{n}$ 。如  $13 \equiv 8 \pmod{5}$  和  $13 \equiv -2 \pmod{5}$ 。

现在定义关系， $R = \{(x, y) | x, y \text{ 是整数, 且 } x \equiv y \pmod{n}\}$ 。  
是等价关系。

- 它划分整数集合为  $n$  类  $C_0, C_1, \dots, C_{n-1}$ ，当且仅当  $x \equiv y \pmod{n}$  时，有  $x \in C_i$  和  $y \in C_i$

## 1.3.1 集合、关系、函数

- 例子
- $R = \{(x, y) | x, y \text{ 是整数, 且 } x \equiv y \pmod{n}\}$ . 是等价关系
- 证明： 设任意  $a, b, c \in I$  (整数)
  - I： 因为  $a - a = k \cdot 0$ ， 所以  $\langle a, a \rangle \in R$
  - II： 若  $a \equiv b \pmod{k}$ ，  $a - b = kt$  ( $t$  为整数)， 则  $b - a = -kt$ ， 所以  $b \equiv a \pmod{k}$
  - III： 若  $a \equiv b \pmod{k}$ ，  $b \equiv c \pmod{k}$ ， 则  $a - b = kt$ ，  $b - c = ks$  ( $t, s$  为整数)，  $a - c = a - b + b - c = k(t + s)$ ， 所以  $a \equiv c \pmod{k}$
- 等价类为： (  $n=3$  时 )
  - $C_0 = \{ \dots, -6, -3, 0, 3, 6, \dots \}$
  - $C_1 = \{ \dots, -5, -2, 1, 4, 7, \dots \}$
  - $C_2 = \{ \dots, -4, -1, 2, 5, 8, \dots \}$

## 1.3.1 集合、关系、函数

- 函数

- 函数  $f$  是一个（二元）关系，就是对于每一个元素  $x \in \text{Dom}(f)$ ，**恰有一个**元素  $y \in \text{Ran}(f)$ ，并且  $(x, y) \in f$ 。在这种情况下，常常把  $(x, y) \in f$  写成  $f(x) = y$ ，并且称  $y$  是  $f$  在  $x$  上的**值或像**。

- 例子

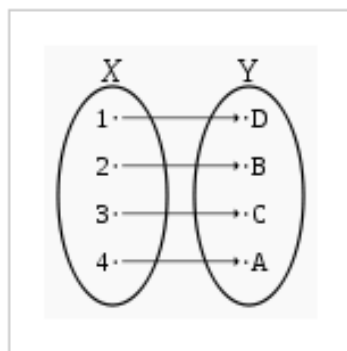
- 关系  $\{(1, 2), (3, 4), (2, 4)\}$  是函数，而关系  $\{(1, 2), (1, 4)\}$  不是。
- 关系  $\{(x, y) | x, y \text{ 是正整数, 且 } x = y^3\}$  是一个函数，而关系  $\{(x, y) | x \text{ 是正整数, } y \text{ 是整数, 且 } x = y^2\}$  不是函数。
- 在例 2.1 中， $R_1$  是函数，而  $R_2$  和  $R_3$  不是。

### 1.3.1 集合、关系、函数

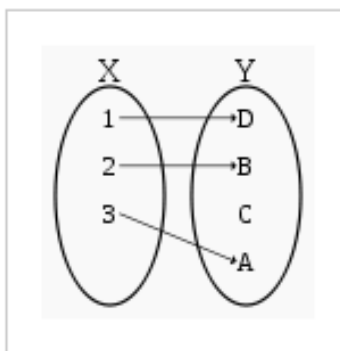
- 设  $f$  是一个函数，对于非空集合  $A$  和  $B$ ，有  $\text{Dom}(f)=A$  和  $\text{Ran}(f)\subseteq B$ ，
  - 如果  $A$  中没有不同元素  $x$  和  $y$ ，使得  $f(x)=f(y)$ ，则称  $f$  是**一对一的**，即  $f(x)=f(y)$  蕴含着  $x=y$ （单射）。
  - 如果  $\text{Ran}(f)=B$ ，则称  **$f$  是映到  $B$  的**（满射）。
  - 如果  $f$  是一对一的，又是映到  $B$  的，则称它是双射的或者称  $A$  和  $B$  之间是**一一对应的**（双射）。

## 1.3.1 集合、关系、函数

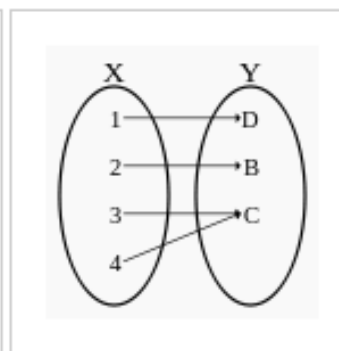
- **单射**：指将不同的变量映射到不同的值的函数。
- **满射**：对于值域中任意元素，都存在至少一个定义域中的元素与之对应。
- **双射**（也称**一一对应**）：既是单射又是满射的函数。直观地说，一个双射函数形成一个对应，并且每一个输入值都有正好一个输出值以及每一个输出值都有正好一个输入值。



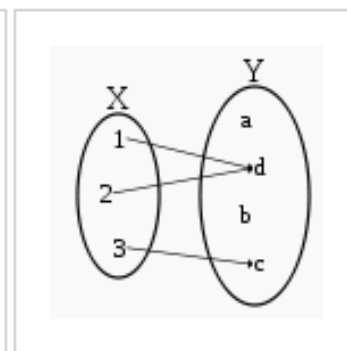
双射（单射与满射）



单射但非满射



满射但非单射



非满射非单射

## 1.3.2 证明方法

- 符号

- 一个命题或断言  $P$  简单地说是一个陈述句，它可以是真或者是假，但不能二者都是
- 符号  $\neg$ ：是**否定符号**。例如， $\neg P$  是命题  $P$  的否定。
- 符号  $\rightarrow$ ：是表示“**蕴含**”。如果  $P \rightarrow Q$ ，就说  $Q$  是  $P$  的必要条件， $P$  是  $Q$  的充分条件。
- $\leftrightarrow$ ：**当且仅当**。语句  $P \leftrightarrow Q$  代表“ $P$  当且仅当  $Q$ ”，也就是“当且仅当  $Q$  为真时  $P$  为真”，可分成  $P \rightarrow Q$  和  $P \leftarrow Q$ 。



## 1.3.2 证明方法——直接证明

- 要证明“ $P \rightarrow Q$ ”，可以先假设  $P$  是真的，然后从  $P$  为真推出  $Q$  为真，这是直接证明法。

- 例子

如果  $n$  是偶数，则  $n^2$  也是偶数。

证明（直接）：

由于  $n$  是偶数，则  $n = 2k$ ， $k$  是某个整数，  
所以有  $n^2 = 4k^2 = 2(2k^2)$ 。

因此，得出  $n^2$  是偶数，证明完毕。

## 1.3.2 证明方法——间接证明

- 蕴含式“ $P \rightarrow Q$ ”逻辑等价于逆反命题“ $\neg Q \rightarrow \neg P$ ”;
- 要证明“ $P \rightarrow Q$ ”，间接等于要证明“ $\neg Q \rightarrow \neg P$ ”.
- 例子

如果  $n^2$  是偶数，那么  $n$  是偶数。

证明（间接）：

首先，我们证明如果  $n$  是奇数，那么  $n^2$  也是奇数。

由于  $n$  是奇数，那么  $n=2k+1$ ， $k$  是某个整数。

那么， $n^2=(2K+1)^2=2(2k^2+2k)+1$ ，

所以  $n^2$  是奇数。

因此，如果  $n^2$  是偶数，那么  $n$  是偶数。

## 1.3.2 证明方法——间接证明

- 为了证明命题“ $P \rightarrow Q$ ”为真，先假定  $P$  为真，但  $Q$  为假。如果这个假设导出矛盾，就意味着假设“ $Q$  为假”必定是错的，所以  $Q$  必定可由  $P$  推出。
- 逻辑因果关系：
  - 如果已知  $P \rightarrow Q$  为真，且  $Q$  为假，则  $P$  必定为假；
  - 所以，如果一开始先假定  $P$  为真， $Q$  为假，从而得出  $P$  为假的结论，这样， $P$  即是真的又是假的，但是  $P$  不能既为真又为假，因此，这是一个矛盾，那么得出  $Q$  为假的假定是错误的，所以最终只有一种可能： $Q$  为真。

## 1.3.2 证明方法——间接证明

- 例子

有无限多的素数。反证法证明：

素数：指在一个大于1的自然数中，除了1和此整数自身外，不能被其他自然数整除的数。

合数：比1大但不是素数的数称为合数。

定义：任何一个合数都可以分解为几个素数的积。

例：100以内的素数有2，3，5，7，11，13，17，19，23，29，31，37，41，43，47，53，59，61，67，71，73，79，83，89，97，在100内共有25个素数。

## 1.3.2 证明方法——间接证明

- 例子

证明（反证法）：（书本）

- 假设相反，仅存在  $K$  个素数  $p_1, p_2, \dots, p_k$ ，这里  $p_1=2, p_2=3, p_3=5$ ，等等。所有其他大于 1 的整数都是合数。
- 令  $n=p_1p_2\cdots p_k+1$ ，令  $p$  为一个素数的因子（注意由前面的假设，由于  $n$  大于  $p_k$ ，所以  $n$  不是素数）。
- 既然  $n$  不是素数，那么  $p_1, p_2, \dots, p_k$  中，必定有一个能够整除  $n$ ，就是说， $p$  是  $p_1, p_2, \dots, p_k$  中的一个，因为  $p$  整除  $p_1p_2\cdots p_k$ 。因此， $p$  整除  $n-p_1p_2\cdots p_k$ 。
- 但是  $n-p_1p_2\cdots p_k=1$ ，由素数的定义可知，因为  $p$  大于 1 所以  $p$  不能整除 1。

这是一个矛盾，于是得到，素数的个数是无限的。

## 1.3.2 证明方法——间接证明

- **反例证明法**用于证明一个假设的命题是错误的，它通常用来证明一个命题在很多情况下是正确的，但是并不永远正确。
- 通常我们用一个反例来证明某个命题是不正确的。
- 例子

令  $f(n)=n^2+n+41$  是定义在非负整数集合上的一个函数。证明断言  $f(n)$  永远是一个素数。例如  $f(0)=41, f(1)=43, \dots$

$f(39)=1601$  都是素数。为了说明该命题有错，只需要找出一个正整数  $n$ ，使得  $f(n)$  是合数即可。

证明：

- 因为  $f(40)=1681=41^2$  是合数，
- 所以该断言为假。

## 1.3.2 证明方法——数学归纳法

- 数学归纳法在证明某一性质对自然数序列  $n_0, n_0+1, n_0+2, \dots$  成立时是一种强有力的方法。
- Steps:
  - 基础步：  
首先证明该性质对于  $n_0$  成立。
  - 归纳步：  
然后证明只要这个性质对于  $n_0, n_0+1, n_0+2, \dots, n-1$  为真，那么必有对于  $n$  这个性质为真



## 1.3.2 证明方法——数学归纳法

- 例子

证明 Bernoulli 不等式：对每一个实数  $x \geq -1$  和每一个自然数  $n$ ，有  $(1+x)^n \geq 1+nx$ 。

- 基础步：如果  $n=1$ ，那么  $1+x \geq 1+x$ 。
- 归纳步：假定不等式对于所有的  $k$  成立， $1 \leq k < n$ , where  $n > 1$ ，那么

$$\begin{aligned}(1+x)^n &= (1+x)(1+x)^{n-1} \\ &= (1+x)(1+(n-1)x) \quad \{by \text{ induction and since } x \geq -1\} \\ &= (1+x)(1+nx-x) \\ &= 1+nx-x+x+nx^2-x^2 \\ &= 1+nx+(nx^2-x^2) \\ &= 1+nx. \quad \{since (nx^2-x^2) \geq 0 \text{ for } n \geq 1\}\end{aligned}$$

- 因此，对于所有的  $n \geq 1$ ，有  $(1+x)^n \geq 1+nx$

### 1.3.3 对数

- 设  $b$  为一个大于 1 的正实数， $x$  是实数，假定对于某个正实数  $y$ ，有  $y=b^x$ 。那么  $x$  称为以  $b$  为底的  $y$  的**对数**，写作：

$$x=\log_b y.$$

这里， $b$  称为**对数的底数**。对于任意大于 0 的实数  $x$  和  $y$ ，有：

$$\log_b xy = \log_b x + \log_b y$$

$$\log_b (c^y) = y \log_b c, c > 0$$

当  $b=2$  时，我们把  $\log_2 x$  写作  $\log x$

当  $b=e$  时，我们把  $\log_e x$  写作  $\ln x$

$$e = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots = 2.718\ 281\ 8\cdots$$

## 1.3.4 底函数、顶函数

令  $x$  是实数

- 用  $\lfloor x \rfloor$  来表示  $x$  的底函数，它被定义为小于等于  $x$  的最大整数； $x$  的顶函数用  $\lceil x \rceil$  来表示，它被定义为大于等于  $x$  的最小整数，例如：

$$\lfloor \sqrt{2} \rfloor = 1, \lceil \sqrt{2} \rceil = 2, \lfloor -2.5 \rfloor = -3, \lceil -2.5 \rceil = -2$$

- 重要等式：
  - $\lceil x/2 \rceil + \lfloor x/2 \rfloor = x$ ;  $\lfloor -x \rfloor = -\lceil x \rceil$ ;  $\lceil -x \rceil = -\lfloor x \rfloor$ .



### 定理 2.1

- $f(x)$  是单调递增函数，使得若  $f(x)$  是整数，则  $x$  是整数。  
那么  $\lfloor f(\lfloor x \rfloor) \rfloor = \lfloor f(x) \rfloor$  且  $\lceil f(\lceil x \rceil) \rceil = \lceil f(x) \rceil$ .



## 1.3.5 阶乘和二项式系数

- 排列：
  - $n$  个不同对象的排列，定义为这些对象排成一排。例如  $a, b, c$  三个元素有 6 种排列，分别是  
 $abc, acb, bac, bca, cab, cba$   
排列与元素的顺序有关。
- 排列数：
  - $n$  个对象中每次取  $k$  个对象的排列数：  
$$P_k^n = n(n-1)\dots(n-k+1) = \frac{n!}{(n-k)!}$$
  - $n$  个对象的排列数： $P_n^n = 1 \times 2 \times \dots \times n$
- 这个量记为： $n! = 1 \times 2 \times \dots \times n$ 
  - $0! = 1$ ；如果  $n \geq 1$ ，则  $n! = n(n-1)!$
  - $n!$  是一个增长非常快的函数

## 1.3.5 阶乘和二项式系数

- 从  $n$  个对象中选取  $k$  个对象的可能方法，通常称为从  $n$  个对象中一次取出  $k$  个对象的**组合**，记做  $C_k^n$ ，组合与顺序无关。
- 例如从字母  $a, b, c, d$  中一次取出 3 个元素的组合有：
  - $abc, abd, acd, bcd,$
- 由于顺序在这里并不重要，从  $n$  个对象中一次取出  $k$  个对象的组合数，等于  $k!$  去除从  $n$  个对象中一次取出  $k$  个对象的排列数，即：

$$C_k^n = \frac{P_k^n}{k!} = \frac{n(n-1)\dots(n-k+1)}{k!} = \frac{n!}{k!(n-k)!}, n \geq k \geq 0$$

$$\binom{n}{k}$$

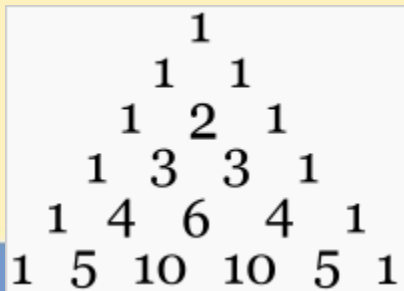
这个如： $\binom{4}{3} = \frac{4!}{3!(4-3)!} = 4$  “从  $n$  选  $k$ ”，称为**二项式系数**。

## 1.3.5 阶乘和二项式系数

- 二项式系数在数学上是二项式定理中的系数族。其必然为正整数，且能以两个非负整数为参数确定，此两参数通常以  $n$  和  $k$  代表，并将二项式系数写作，亦即是二项式幂  $(1+x)^n$  的多项式展式中， $x^k$  项的系数。如下：

$$(1+x)^n = C(n,0) + C(n,1)x + C(n,2)x^2 + \dots + C(n,k)x^k + \dots + C(n,n-1)x^{(n-1)} + C(n,n)x^n$$
$$= \sum_{j=0}^n \binom{n}{j} x^j$$

- 如将二项式系数的  $n$  值顺序排列成行，每行为  $k$  值由 0 至  $n$  列出，则构成帕斯卡三角形。



|   |   |    |    |   |   |
|---|---|----|----|---|---|
|   |   | 1  |    |   |   |
|   | 1 |    | 1  |   |   |
|   | 1 | 2  | 1  |   |   |
|   | 1 | 3  | 3  | 1 |   |
|   | 1 | 4  | 6  | 4 | 1 |
| 1 | 5 | 10 | 10 | 5 | 1 |

## 1.3.5 阶乘和二项式系数

- 特点：（ P47 ）

$$\binom{n}{k} = \binom{n}{n-k}, \text{ 特别的 } \binom{n}{n} = \binom{n}{0} = 1$$

递归公式 
$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

- 定理 2.2

- 设  $n$  为正整数，那么：

$$(1+x)^n = \sum_{j=0}^n \binom{n}{j} x^j$$

X=1

$$\binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{n} = 2^n$$

X=-1

$$\binom{n}{0} - \binom{n}{1} + \binom{n}{1} - \cdots \pm \binom{n}{n} = 0$$



## 1.3.6 鸽巢原理

组合数学中解决计数问题的一个工具。

- 假定一群鸽子飞入一组巢安歇，如果鸽子比鸽巢多，那么一定至少有一个鸽巢里有两只或两只以上的鸽子。
- 这个原理除了鸽子和鸽巢外，也可用于其他对象，因此也称为（狄利克莱，德国，19 世纪）**抽屉原理**、**鞋盒原理**。

10 只鸽子放进 9 个鸽笼，那么一定有一个鸽笼放进了至少两只鸽子



## 1.3.6 鸽巢原理

**简单形式：**如果  $K+1$  个或更多的物体放入  $K$  个盒子，那么至少有一个盒子含 2 个或更多的物体。

例 1：在 13 个人中存在两个人，他们的生日在同一个月里

例 2：设有  $n$  对已婚夫妇。为保证有一对夫妇被选出，至少要从这  $2n$  个人中选出多少人？（ $n+1$ ）

## 1.3.6 鸽巢原理

- 定理 2.3 (推广的鸽巢原理)

如果把  $n$  个球分别放在  $m$  个盒子中, 那么:

- 存在一个盒子, 必定至少装  $\lceil n/m \rceil$  个球;
- 存在一个盒子, 必定最多装  $\lfloor n/m \rfloor$  个球。

等同于: 当盒子仅有  $n$  个, 而球的数目大于  $n*m$  时, 则必有一个盒子中至少有  $m+1$  或多于  $m+1$  个物体。

例: 在 100 人中至少有  $\lceil 100/12 \rceil = 9$  个人出生在同一个月。

一定有一个月, 最多只有  $\lfloor 100/12 \rfloor = 8$  个人出生。(即不可能每个月都有 9 个人出生)

- 例子

令  $G = (V, E)$  是一个连通的无向图, 有  $m$  个顶点。令  $p$  是  $G$  中访问  $n > m$  个顶点的一条路径, 那么  $p$  必定包含一条回路。

证明: 由于  $\lceil n/m \rceil > 1$ , 至少有一顶点, 假设是  $v$ , 它将被  $p$  访问超过一次, 这样, 这条路径以  $v$  开始又以  $v$  结束的部分形成了一条回路。

### 1.3.7 和式

$$\sum_{j=1}^n j = \frac{n(n+1)}{2} = \Theta(n^2)$$

$$\sum_{j=1}^n j^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$$

P48

## 1.3.7 和式

- 令  $f(x)$  是一个单调递减或单调递增的连续函数，现在来估算和式的值。
$$\sum_{j=1}^n f(j)$$

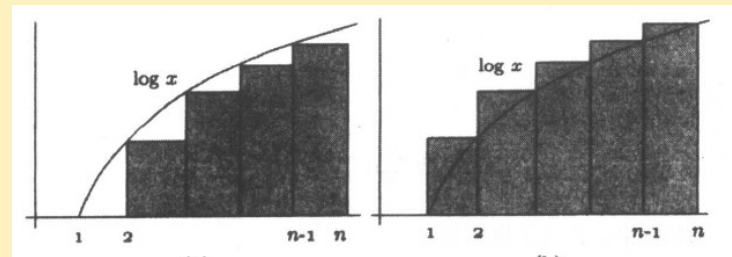
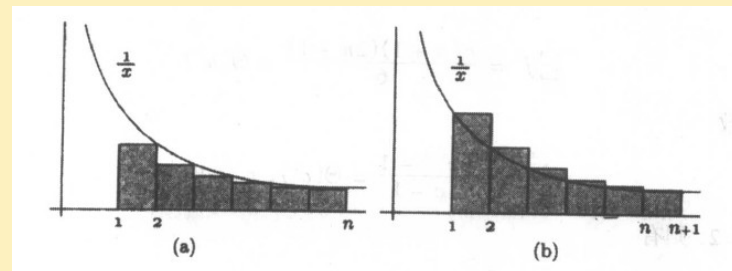
可以通过积分来近似和式，得出上下界如下：

- 如果  $f(x)$  是递减的，那么有

$$\int_m^{n+1} f(x)dx \leq \sum_{j=m}^n f(j) \leq \int_{m-1}^n f(x)dx$$

- 如果  $f(x)$  是递增的，那么有

$$\int_{m-1}^n f(x)dx \leq \sum_{j=m}^n f(j) \leq \int_m^{n+1} f(x)dx$$



### 1.3.8 递推关系

- 递归公式是用它自身来定义的一个公式，在这种情况下，我们称这个定义为递推关系或递推式。例如，正奇数序列可以用递推式描述如下：



$$f(n)=f(n-1)+2, \text{ if } n>1 \text{ and } f(1)=1.$$

- A 递推关系如果具有如下这种形式，称为常系数线性齐次递推式

$$f(n)=a_1f(n-1)+a_2f(n-2)+\dots+a_kf(n-k).$$

这里， $f(n)$  称为是  $k$  次的。当一个附加项包括常量或者  $n$  的函数出现在递推中时，那么它就称为非齐次的。





## 1.3.8 递推关系

- 定义：数列  $\{f(n)\}$  满足递推关系
- $f(n)=a_1f(n-1)+a_2f(n-2)+\cdots+a_kf(n-k)$ ,  $a_i$  为常数,  $i=1,2,\cdots,k, n\geq k$ ,  $a_k\neq 0$ , 称该递推关系为  $f(n)$  的  **$k$  阶常系数线性齐次递推关系**。
- 形如  $f(n)=a_1f(n-1)+a_2f(n-2)+\cdots+a_kf(n-k)+g(n)$ ,  $a_i$  为常数,  $i=1,2,\cdots,k, n\geq k$ ,  $a_k\neq 0$ , 称该递推关系为  $f(n)$  的  **$k$  阶常系数线性非齐次递推关系**。
- 方程  $x^k-a_1x^{k-1}-a_2x^{k-2}-\cdots-a_k=0$  称为  $k$  阶常系数线性齐次递推关系的 **特征方程**, 它的  $k$  个根  $r_1, r_2, \dots, r_k$  称为该递推关系的 **特征根**。
- 若  $k$  阶常系数线性齐次递推关系的特征方程有  $k$  个互异的特征根:  $r_1, r_2, \dots, r_k$ , 则该递推关系的 **通解** 为:  **$f(n)=c_1r_1^n+c_2r_2^n+\cdots+c_kr_k^n$** , 其中  $c_1, c_2, \dots, c_k$  为任意常数。



### 1.3.8 递推关系

- 递推关系是离散变量之间变化规律中常见的一种方式，与生成函数一样是解决计数问题的有力工具。
- 数列  $\{f(n)\}$ ，如从某项后， $f(n)$  前  $k$  项可推出  $f(n)$  的普遍规律，就称为递推关系。
- 利用递推关系和初值在某些情况下可以求出序列的通项表达式  $f(n)$ ，称为该递推关系的解。
- 不是所有的递推关系都可求出其解，只是某些特殊类型有成熟解法。

## 1.3.8 递推关系——线性齐次递推式的求解

$$x^n = a_1 x^{n-1} + a_2 x^{n-2} + \cdots + a_k x^{n-k}$$

两边同时除以  $x^{n-k}$  得到

$$x^k = a_1 x^{k-1} + a_2 x^{k-2} + \cdots + a_k$$

p53

或者写成

$$x^k - a_1 x^{k-1} - a_2 x^{k-2} - \cdots - a_k = 0 \quad (2.20)$$

等式 2.20 称为递推关系 2.19 的特征方程。

下面，我们把注意力限于一阶和二阶线性齐次递推关系。一阶齐次递推方程的解可以直接得到，令  $f(n) = af(n-1)$ ，假定序列从  $f(0)$  开始，由于

$$f(n) = af(n-1) = a^2 f(n-2) = \cdots = a^n f(0)$$

容易看到  $f(n) = a^n f(0)$  是递推的解。

如果递推的次数是 2，那么特征方程变成  $x^2 - a_1 x - a_2 = 0$ ，令这个二次方程的根是  $r_1$  和  $r_2$ ，递推的解是

$$f(n) = c_1 r_1^n + c_2 r_2^n, \text{ 如果 } r_1 \neq r_2; f(n) = c_1 r^n + c_2 nr^n, \text{ 如果 } r_1 = r_2 = r$$

这里  $c_1$  和  $c_2$  是序列的初值  $f(n_0)$  和  $f(n_0 + 1)$ 。

## 1.3.8 递推关系——线性齐次递推式的求解

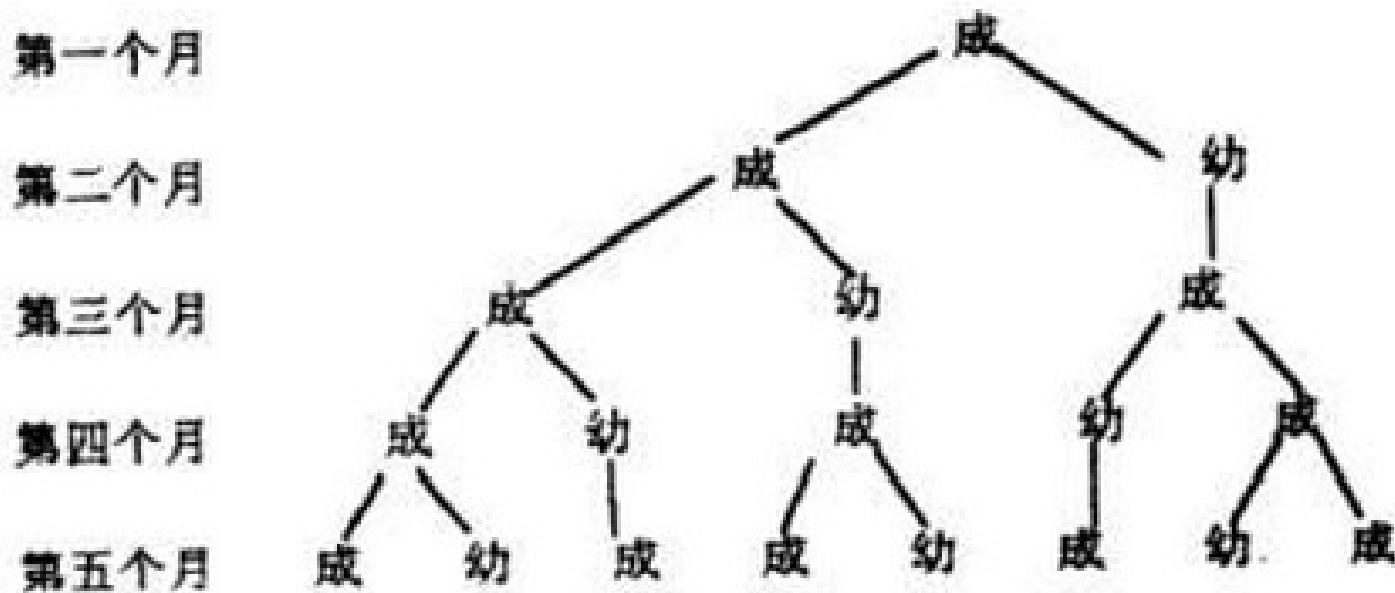
例 2.18 考虑序列 1, 4, 16, 64, 256, ... 可以用递推关系表示为  $f(n) = 3f(n-1) + 4f(n-2)$ , 且  $f(0) = 1, f(1) = 4$ , 特征方程是  $x^2 - 3x - 4 = 0$ , 有  $r_1 = -1, r_2 = 4$ 。这样递推的解是  $f(n) = c_1(-1)^n + c_24^n$ 。为求出  $c_1$  和  $c_2$  的值, 求解下面两个联立方程

$$f(0) = 1 = c_1 + c_2, f(1) = 4 = -c_1 + 4c_2$$

得到:  $c_1 = 0, c_2 = 1$ , 所以  $f(n) = 4^n$ 。

## 1.3.8 递推关系

例：13 世纪初意大利数学家 Fibonacci 研究过著名的兔子繁殖数目问题：设一对一雌一雄小兔刚满 2 个月时，便可繁殖出一雌一雄一对小兔。以后每隔 1 个月生一对一雌一雄小兔。由一对刚出生的小兔开始饲养到第  $n$  个月，有多少对兔子？



例 2.20

## 1.3.8 递推关系——Fibonacci 序列

例 2.20 考虑 Fibonacci 序列  $1, 1, 2, 3, 5, 8, \dots$  它可以用递推关系表示为  $f(n) = f(n-1) + f(n-2)$ , 且  $f(1) = f(2) = 1$ 。为了简化解, 我们可以引进额外项  $f(0) = 0$ 。特征方程是  $x^2 - x - 1 = 0$ ,  $r_1 = (1 + \sqrt{5})/2$ ,  $r_2 = (1 - \sqrt{5})/2$ 。这样递推关系的解是

$$f(n) = c_1 \left( \frac{1 + \sqrt{5}}{2} \right)^n + c_2 \left( \frac{1 - \sqrt{5}}{2} \right)^n$$

为求出  $c_1$  和  $c_2$  的值, 求解下面两个联立方程

$$f(0) = 0 = c_1 + c_2, \quad f(1) = 1 = c_1 \left( \frac{1 + \sqrt{5}}{2} \right) + c_2 \left( \frac{1 - \sqrt{5}}{2} \right)$$

求解的结果, 我们得到:  $c_1 = 1/\sqrt{5}$ ,  $c_2 = -1/\sqrt{5}$ , 所以

$$f(n) = \frac{1}{\sqrt{5}} \left( \frac{1 + \sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left( \frac{1 - \sqrt{5}}{2} \right)^n$$

因为  $(1 - \sqrt{5})/2 \approx -0.618\,034$ , 当  $n$  很大时第二项趋近于 0, 所以当  $n$  足够大时

$$f(n) \approx \frac{1}{\sqrt{5}} \left( \frac{1 + \sqrt{5}}{2} \right)^n \approx 0.447\,214 (1.618\,03)^n$$

数值  $\phi = (1 + \sqrt{5})/2 \approx 1.618\,03$ , 称为黄金率。在例 2.11 中, 我们已经证明了对于所有的  $n \geq 1$ ,  $f(n) \leq \phi^{n-1}$ 。

## 1.3.8 递推关系——非齐次递推关系的解

- 递推式  $f(n)=f(n-1)+g(n)$  的解： $f(n) = f(0) + \sum_{i=1}^n g(i)$  (P54)

- 递推式  $f(n)=g(n)f(n-1), n \geq 1$  的解： $f(n) = g(n)g(n-1)\dots g(1)f(0)$

- 递推式  $f(n)=g(n)f(n-1)+h(n), n \geq 1$  的解：

$$f(n) = g(n)g(n-1)\dots g(1) \left( f(0) + \sum_{i=1}^n \frac{h(i)}{g(i)g(i-1)\dots g(1)} \right), n \geq 1$$



遗憾的是,一般来说没有容易的办法处理非齐次递推关系。这里,我们将限于在算法分析中经常用到的一些基本的非齐次递推关系上。也许最简单的非齐次递归关系是

$$\underline{f(n) = f(n-1) + g(n), n \geq 1} \quad (2.21)$$

其中  $g(n)$  是另一个序列。容易看到递推式 2.21 的解是

$$\longrightarrow f(n) = f(0) + \sum_{i=1}^n g(i)$$

例如,递推  $f(n) = f(n-1) + 1$ , 且  $f(0) = 0$  的解是  $f(n) = n$ 。

现在来考虑齐次递推关系

$$\underline{f(n) = g(n)f(n-1), n \geq 1} \quad (2.22)$$

这也容易看出递推关系 2.22 的解是

$$\longrightarrow f(n) = g(n)g(n-1)\cdots g(1)f(0)$$

例如,递推式  $f(n) = nf(n-1)$ , 且  $f(0) = 1$  的解是  $f(n) = n!$ 。

下面考虑非齐次递推关系

$$\underline{f(n) = g(n)f(n-1) + h(n), n \geq 1} \quad (2.23)$$

其中  $h(n)$  又是另一个序列。定义一个新函数  $f'(n)$  如下, 令



$$f(n) = g(n)f(n-1) + h(n), n \geq 1 \quad (2.23)$$

其中  $h(n)$  又是另一个序列。定义一个新函数  $f'(n)$  如下, 令

$$f(n) = g(n)g(n-1)\cdots g(1)f'(n), n \geq 1; f'(0) = f(0)$$

将递推 2.23 中的  $f(n)$  和  $f(n-1)$  代入相应的式子, 得到

$$g(n)g(n-1)\cdots g(1)f'(n) = g(n)(g(n-1)\cdots g(1)f'(n-1)) + h(n)$$

它简化为

$$f'(n) = f'(n-1) + \frac{h(n)}{g(n)g(n-1)\cdots g(1)}, n \geq 1$$

因此

$$f'(n) = f'(0) + \sum_{i=1}^n \frac{h(i)}{g(i)g(i-1)\cdots g(1)}, n \geq 1$$

得出

$$\longrightarrow f(n) = g(n)g(n-1)\cdots g(1)\left(f(0) + \sum_{i=1}^n \frac{h(i)}{g(i)g(i-1)\cdots g(1)}\right), n \geq 1 \quad (2.24)$$

## 1.3.8 递推关系——分治递推关系的解

- 递推式：

$$f(n) = \begin{cases} d & \text{if } n \leq n_0 \\ a_1 f(n/c_1) + a_2 f(n/c_2) + \dots + a_p f(n/c_p) + g(n) & \text{if } n > n_0 \end{cases}$$

这里， $a_1, a_2, \dots, a_p, c_1, c_2, \dots, c_p$  和  $n_0$  都是非负整数， $d$  是一个非负常数， $p \geq 1$ ， $g(n)$  是从非负整数集合到实数集合的一个函数，这里我们讨论三种最普通的求解分治递推式的技术。

- 三种求解分治递推式的技术：
  - 展开递推式
  - 代入法
  - 更换变元

## 1.3.8 递推关系——展开递推式

- 通过显而易见的方式将其反复展开
- 这种方法，在函数定义的比率不相等的递归式中，很难使用
- 例 2.23



- 考虑递推式

$$f(n) = \begin{cases} d & \text{if } n = 1 \\ 2f(n/2) + bn \log n & \text{if } n \geq 2 \end{cases}$$

这里  $b$  和  $d$  是非负常数， $n$  是 2 的幂。

见 p56.

## 1.3.8 递推关系——展开递推式

例 2.23 考虑递推式

$$f(n) = \begin{cases} d & \text{若 } n = 1 \\ 2f(n/2) + bn \log n & \text{若 } n \geq 2 \end{cases}$$

这里  $b$  和  $d$  是非负常数,  $n$  是 2 的幂。我们用下面的方法来求解递推式(设  $k = \log n$ )。

$$\begin{aligned} f(n) &= 2f(n/2) + bn \log n \\ &= 2(2f(n/2^2) + b(n/2)\log(n/2)) + bn \log n \\ &= 2^2 f(n/2^2) + bn \log(n/2) + bn \log n \\ &= 2^2 (2f(n/2^3) + b(n/2^2)\log(n/2^2)) + bn \log(n/2) + bn \log n \\ &= 2^3 f(n/2^3) + bn \log(n/2^2) + bn \log(n/2) + bn \log n \\ &\vdots \\ &= 2^k f(n/2^k) + bn(\log(n/2^{k-1}) + \log(n/2^{k-2}) + \cdots + \log(n/2^{k-k})) \\ &= dn + bn(\log 2^1 + \log 2^2 + \cdots + \log 2^k) \\ &= dn + bn \sum_{j=1}^k \log 2^j \\ &= dn + bn \sum_{j=1}^k j \\ &= dn + bn \frac{k(k+1)}{2} \\ &= dn + \frac{bn \log^2 n}{2} + \frac{bn \log n}{2} \end{aligned}$$

## 1.3.8 递推关系——展开递推式

### 引理 2.1

- 设  $a$  和  $c$  是非负整数,  $b, d, x$  是非负常数, 并且对于某个非负整数  $k$ , 令  $n=c^k$ , 那么, 下面递推式

$$f(n) = \begin{cases} d & \text{if } n=1 \\ af(n/c)+bn^x & \text{if } n \geq 2 \end{cases}$$

的解是

$$f(n) = \begin{cases} bn^x \log_c n + dn^x & \text{if } a = c^x \\ \left( d + \frac{bc^x}{a - c^x} \right) n^{\log_c a} - \left( \frac{bc^x}{a - c^x} \right) n^x & \text{if } a \neq c^x \end{cases}$$

## 1.3.8 递推关系——展开递推式



### 推论 2.1

- 设  $a$  和  $c$  是非负整数， $b, d, x$  是非负常数，并且对于某个非负整数  $k$ ，令  $n=c^k$ ，那么，下面递推式

$$f(n) = \begin{cases} d & \text{if } n=1 \\ af(n/c)+bn^x & \text{if } n \geq 2 \end{cases}$$

的解满足

$$f(n) = bn^x \log_c n + dn^x \quad \text{if } a = c^x$$

$$f(n) \leq \left(\frac{bc^x}{c^x - a}\right)n^x \quad \text{if } a < c^x$$

$$f(n) \leq \left(d + \frac{bc^x}{a - c^x}\right)n^{\log_c a} \quad \text{if } a > c^x$$





## 1.3.8 递推关系——展开递推式



### 推论 2.2

- 设  $a$  和  $c$  是非负整数， $b, d, x$  是非负常数，并且对于某个非负整数  $k$ ，令  $n=c^k$ ，那么，下面递推式

$$f(n) = \begin{cases} d & \text{if } n=1 \\ af(n/c)+bn & \text{if } n \geq 2 \end{cases}$$

的解是

$$f(n) = bn \log_c n + dn \quad \text{if } a = c$$

$$f(n) = \left(d + \frac{bc}{a-c}\right)n^{\log_c a} - \left(\frac{bc}{a-c}\right)n \quad \text{if } a \neq c$$





## 1.3.8 递推关系——展开递推式



### 定理 2.5

- 设  $a$  和  $c$  是非负整数,  $b, d, x$  是非负常数, 并且对于某个非负整数  $k$ , 令  $n=c^k$ , 那么, 下面递推式

$$f(n) = \begin{cases} d & \text{if } n=1 \\ af(n/c)+bn^x & \text{if } n \geq 2 \end{cases}$$

的解是

$$f(n) = \begin{cases} \Theta(n^x) & \text{if } a < c^x \\ \Theta(n^x \log n) & \text{if } a = c^x \\ \Theta(n^{\log_c a}) & \text{if } a > c^x \end{cases}$$

特别的, 如果  $x=1$ , 那么

$$f(n) = \begin{cases} \Theta(n) & \text{if } a < c \\ \Theta(n \log n) & \text{if } a = c \\ \Theta(n^{\log_c a}) & \text{if } a > c \end{cases}$$



### 1.3.8 递推关系——代入法

- 这种方法通常用来证明上下界，它也能用来证明精确解。
- 在这种方法中，我们猜想一个解，并尝试用数学归纳法来证明，与在归纳法证明中常用的做法不同的是，这里我们先证明带有一个或多个常量的归纳步，当  $n$  为任意值时，一旦对  $f(n)$  的断言成立，为了使解适用于一个和多个边界条件，如果需要的话，我们再来调整常量。

## 1.3.8 递推关系——代入法

### 例子

- 考虑下面递推式

$$f(n) = \begin{cases} d & \text{if } n = 1 \\ f(\lfloor n/2 \rfloor) + f(\lceil n/2 \rceil) + bn & \text{if } n \geq 2 \end{cases}$$

其中  $b, d$  是非负常数。当  $n$  是 2 的幂时，递推式可以简化为

$$f(n) = 2f(n/2) + bn,$$

由推论 2.2，它的解是  $bn \log n + dn$ ，因此，我们做一个猜测，对于某个常数  $c > 0$ ， $f(n) \leq cbn \log n + dn$ ，后面再来确定  $c$  的值。

见 P59.

例 2.24 考虑下面递推式

$$f(n) = \begin{cases} d & \text{若 } n = 1 \\ f(\lfloor n/2 \rfloor) + f(\lceil n/2 \rceil) + bn & \text{若 } n \geq 2 \end{cases}$$

其中  $b, d$  是非负常数。当  $n$  是 2 的幂时，递推式可以简化为

$$f(n) = 2f(n/2) + bn$$

由推论 2.2，它的解是  $bn \log n + dn$ ，因此，我们做一个猜测，对于某个常数  $c > 0$ ， $f(n) \leq cbn \log n + dn$ ，后面再来确定  $c$  的值。假定这个断言在  $n \geq 2$  时对于  $\lfloor n/2 \rfloor$  和  $\lceil n/2 \rceil$  都成立，在递推式中代入  $f(n)$ ，可以得出

$$\begin{aligned} f(n) &= f(\lfloor n/2 \rfloor) + f(\lceil n/2 \rceil) + bn \\ &\leq cb\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + d\lfloor n/2 \rfloor + cb\lceil n/2 \rceil \log \lceil n/2 \rceil + d\lceil n/2 \rceil + bn \\ &\leq cb\lfloor n/2 \rfloor \log \lceil n/2 \rceil + cb\lceil n/2 \rceil \log \lceil n/2 \rceil + dn + bn \\ &= cbn \log \lceil n/2 \rceil + dn + bn \\ &\leq cbn \log((n+1)/2) + dn + bn \\ &= cbn \log(n+1) - cbn + dn + bn \end{aligned}$$

为了证明  $f(n)$  最大为  $cn \log n + dn$ ，必定有  $cbn \log(n+1) - cbn + bn \leq cn \log n$ ，也就是  $c \log(n+1) - c + 1 \leq c \log n$ ，化简为

$$c \geq \frac{1}{1 + \log n - \log(n+1)} = \frac{1}{1 + \log \frac{n}{n+1}}$$

## 1.3.8 递推关系——代入法



定理 2.6

• 设

$$f(n) = \begin{cases} d & \text{if } n=1 \\ f(\lfloor n/2 \rfloor) + f(\lceil n/2 \rceil) + bn & \text{if } n \geq 2 \end{cases}$$

b 和 d 是非负常数，那么

$$f(n) = \Theta(n \log n)$$

## 1.3.8 递推关系——代入法

### 例子

- 考虑下面的递推式

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ b & \text{if } n = 1 \\ f(\lfloor c_1 n \rfloor) + f(\lceil c_2 n \rceil) + bn & \text{if } n \geq 2 \end{cases}$$

其中  $b, c_1, c_2$  是正常数, 且  $c_1 + c_2 = 1$ , 当  $c_1 = c_2 = 1/2$ ,  $n$  是 2 的幂时, 上述递推式可以化简为

$$f(n) = \begin{cases} b & \text{if } n = 1 \\ 2f(n/2) + bn & \text{if } n \geq 2 \end{cases}$$

由推论 2.2, 它的解是  $bn \log n + bn$ , 因此我们可以猜测, 对于某个常数  $c > 0$ ,  $f(n) \leq cbn \log n + bn$ ,  $c$  的值后面来确定.

见 p56.

## 1.3.8 递推关系——代入法



### 定理 2.7

- 设  $b, c_1, c_2$  是非负常数, 那么递推式

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ b & \text{if } n = 1 \\ f(\lfloor c_1 n \rfloor) + f(\lfloor c_2 n \rfloor) + bn & \text{if } n \geq 2 \end{cases}$$

的解是

$$f(n) = \begin{cases} O(n \log n) & \text{if } (c_1 + c_2 = 1) \\ \Theta(n) & \text{if } (c_1 + c_2 < 1) \end{cases}$$



### 1.3.8 递推关系——更换变元

- 在有些递推式中，如果改变函数的定义域，在新的定义域中定义一个新的易于求解的递推关系，这样会更加方便。

## 1.3.8 递推关系——更换变元

- 例子
  - 考虑递推式

$$f(n) = \begin{cases} 1 & \text{if } n = 2 \\ 1 & \text{if } n = 4 \\ f(n/2) + f(n/4) & \text{if } n > 4 \end{cases}$$

$n$  是 2 的幂

见 P62

这里假定  $n$  是 2 的幂, 令  $g(k) = f(2^k)$ ,  $k = \log n$ , 那么

$$g(k) = \begin{cases} 1 & \text{若 } k = 1 \\ 1 & \text{若 } k = 2 \\ g(k-1) + g(k-2) & \text{若 } k > 2 \end{cases}$$

$g(k)$  就是例 2.20 中讨论过的 Fibonacci 递推式, 它的解是

$$g(k) = \frac{1}{\sqrt{5}} \left( \frac{1+\sqrt{5}}{2} \right)^k - \frac{1}{\sqrt{5}} \left( \frac{1-\sqrt{5}}{2} \right)^k$$

### 例 2.28 考虑递推式

$$f(n) = \begin{cases} d & \text{若 } n = 1 \\ 2f(n/2) + bn \log n & \text{若 } n \geq 2 \end{cases}$$

我们已在例 2.23 中用展开递推法求出解了。这里  $n$  是 2 的幂，因此令  $k = \log n$ ，即  $n = 2^k$ ，这样递推式可以重写成

$$f(2^k) = \begin{cases} d & \text{若 } k = 0 \\ 2f(2^{k-1}) + bk2^k & \text{若 } k \geq 1 \end{cases}$$

现在令  $g(k) = f(2^k)$ ，就有

$$g(k) = \begin{cases} d & \text{若 } k = 0 \\ 2g(k-1) + bk2^k & \text{若 } k \geq 1 \end{cases}$$

这个递推式具有等式 2.23 的形式，所以可用 2.8.2 节的方法来求解递推式。令

$$2^k h(k) = g(k), \text{ 且 } h(0) = g(0) = d$$

那么

$$2^k h(k) = 2(2^{k-1} h(k-1)) + bk2^k$$

或

$$h(k) = h(k-1) + bk$$

递推式的解是

$$h(k) = h(0) + \sum_{j=1}^k bj = d + \frac{bk(k+1)}{2}$$

因此

$$g(k) = 2^k h(k) = d2^k + \frac{bk^2 2^k}{2} + \frac{bk2^k}{2} = dn + \frac{bn \log^2 n}{2} + \frac{bn \log n}{2}$$

它和例 2.23 中得到的解相同。



# 主要内容



1.1 算法的基本概念

1.2 算法分析基础

1.3 算法分析常用的数学知识

1.4 数据结构的表示、操作算法及分析



# 本章内容



## 1.4 数据结构

1.4.1 基本概念和术语

1.4.2 逻辑结构和存储结构

1.4.3 链表

1.4.4 堆栈和队列

1.4.5 图

1.4.6 树、根树、二叉树

1.4.7 赫夫曼树及其应用

1.4.8 最短路径



## 1.4.1 基本概念和术语

- **数据 (Data):** 是对客观事物的符号表示。在计算机科学中是指所有能输入到计算机中并被计算机程序处理的符号的总称。
- **数据元素 (Data Element):** 数据的基本单位。在计算机程序中通常作为一个整体进行考虑和处理。一个数据元素可由若干个数据项组成。数据项是数据的不可分割的最小单位。

| 数据元素  |       |       |       |
|-------|-------|-------|-------|
| 数据项   |       | 数据项   |       |
| 001   | 高等数学  | 樊映川   | S01   |
| 002   | 理论力学  | 罗远祥   | L01   |
| 003   | 高等数学  | 华罗庚   | S01   |
| 004   | 线性代数  | 栾汝书   | S02   |
| ..... | ..... | ..... | ..... |



## 1.4.1 基本概念和术语

- **数据对象 (Data Object)**：是性质相同的数据元素的集合。是数据的一个子集。
- **数据结构 (Data Structure)**：是相互之间存在一种或多种特定关系的数据元素的集合。

数据结构包括逻辑结构、物理结构和施加在数据上的运算。

例如：

3214,6587,9345 — a1(3214),a2(6587),a3(9345)

则在数据元素 a1、a2 和 a3 之间存在着“次序”关系  $\square a1,a2>$ 、

$\square a2,a3>$   
 $\begin{matrix} 3214, & 6587, & & 6587, & 3214, & 9345 \\ & a1 & a2 & a2 & a1 & a3 \end{matrix} \neq$

## 数据结构的三个方面：

数据的逻辑结构

线性结构

线性表

栈

队列

非线性结构

树形结构

图状结构

数据的存储结构

顺序存储

链式存储

数据的运算：检索、排序、插入、删除、修改等

## 1.4.2 逻辑结构和存储结构

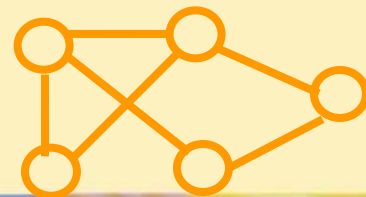
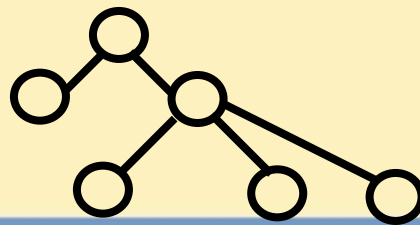
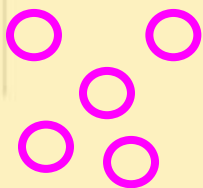
• **逻辑结构**：是对数据元素之间逻辑关系的描述。根据数据元素之间的逻辑结构可将数据结构分为四类：

集合——数据元素除了同属于一种类型外，别无其它关系。

线性结构——数据元素之间存在一对一的关系。如线性表、栈、队列。

树形结构——数据元素之间存在一对多的关系。如树。

图状结构——数据元素之间存在多对多的关系，如图。



## 1.4.2 逻辑结构和存储结构

数据元素之间的关系在计算机中有两种不同的表示方法：

顺序映像和非顺序映像。分别对应两种存储结构：

**顺序存储结构**：借助元素在存储器中的相对位置来表示数据元素间的逻辑关系。

**链式存储结构**：借助指示元素存储地址的指针表示数据元素间的逻辑关系。

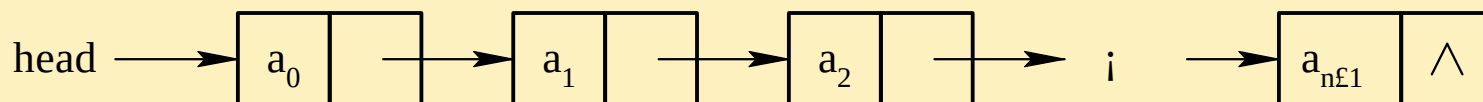
数据的逻辑结构与存储结构密切相关：

算法设计 —— 逻辑结构

算法实现 —— 存储结构

## 1.4.3 链表

- 链表由有限的元素或节点序列组成，节点包含信息和到另一个节点的链。
- 如果节点  $x$  指向节点  $y$ ，那么  $x$  称为  $y$  的前驱节点， $y$  称为  $x$  的后续节点。
- 指向第一个元素的链称为表头



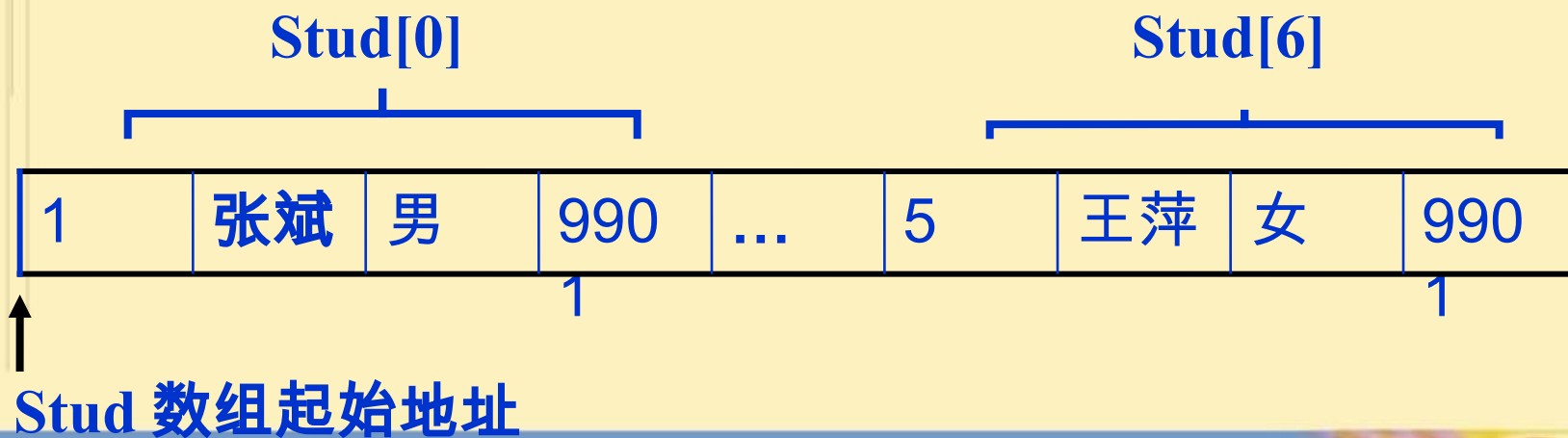
### 1.4.3 链表

存放学生表的结构体数组 Stud 定义为:

```
struct
{
 int no; /* 存储学号 */
 char name[8]; /* 存储姓名 */
 char sex[2]; /* 存储性别 */
 char class[4]; /* 存储班号 */
} Stud[7]={ {1,“ 张斌”,“ 男”,“9901”},...,
 {5,“ 王萍 ”,“ 女 ”,“9901”}};
```

### 1.4.3 链表

结构体数组 Stud 各元素在内存中顺序存放，即第  $i(1 \leq i \leq 6)$  个学生对应的元素 Stud[i] 存放在第  $i+1$  个学生对应的元素 Stud[i+1] 之前，Stud[i+1] 正好在 Stud[i] 之后。





### 1.4.3 链表

存放学生表的链表的结点类型 StudType 定义为：

```
typedef struct studnode
{
 int no; /* 存储学号 */
 char name[8]; /* 存储姓名 */
 char sex[2]; /* 存储性别 */
 char class[4]; /* 存储班号 */
 struct studnode *next; /* 存储指向下一个学生的指针 */
} StudType ;
```

### 1.4.3 链表

链表首结点地址 head



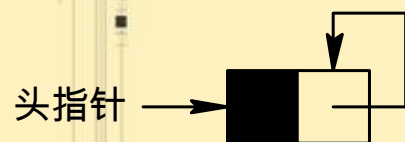
学生表构成的链表如右图所示。其中的 head 为第一个数据元素的指针。

学生表构成的链表

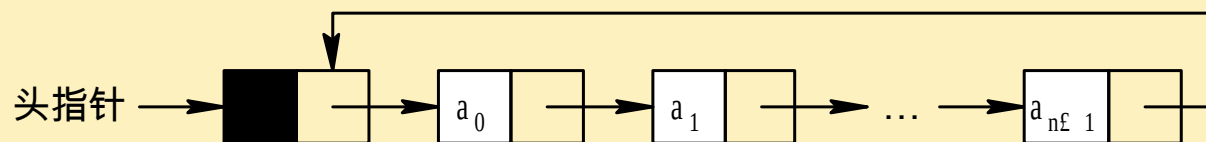
### 1.4.3 链表

- 如果链表中存在由最后一个元素到第一个元素的链，那么这种链表称为**循环链表**
- 如果在一个链表中每一个节点（第一个节点可能除外）也指向它的前驱节点，那么这个链表称为**双向链表**；
- 如果第一个和最后一个节点也被一对链连接起来，这就是一个**循环双向链表**。

## 1.4.3 链表



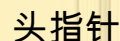
( a )



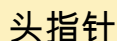
( b )

带头结点的循环链表

(a) 空链； (b) 非空链



( a )

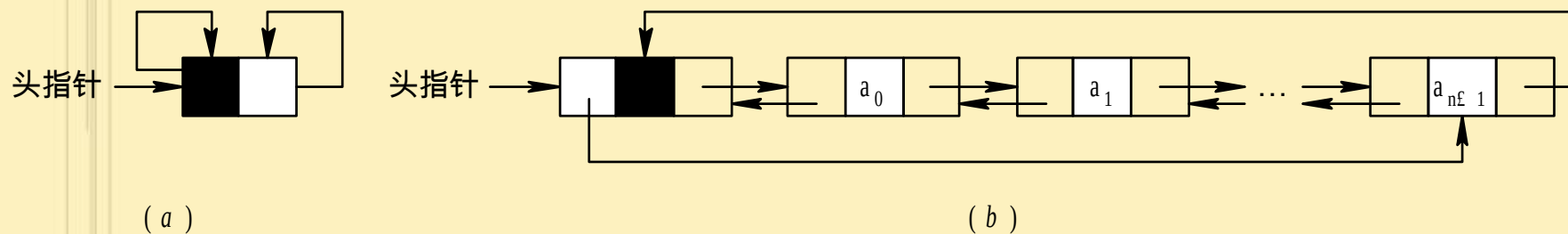


( b )

## 带头结点的双向链表

(a) 空链；      (b) 非空链

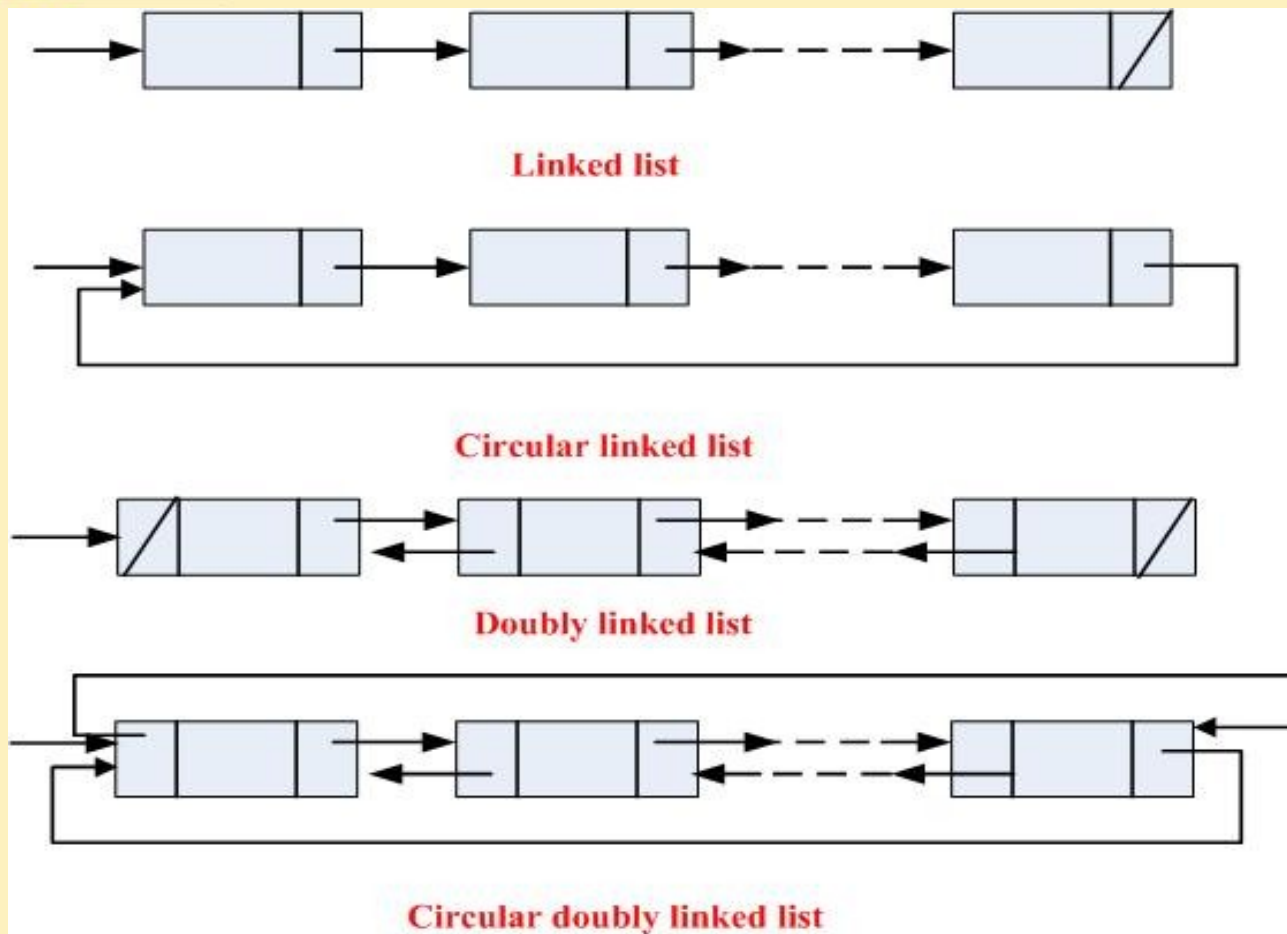
## 1.4.3 链表



带头结点的循环双向链表

(a) 空链； (b) 非空链

## 1.4.3 链表





### 1.4.3 链表——2 个主要操作

- 插入
- 删除
- 与数组不同，在链表中插入和删除元素只花费**固定的时间**。
- 对链表的访问加入一些限制，就可以得到两种基础性的数据结构：堆栈和队列

## 1.4.3 链表——插入

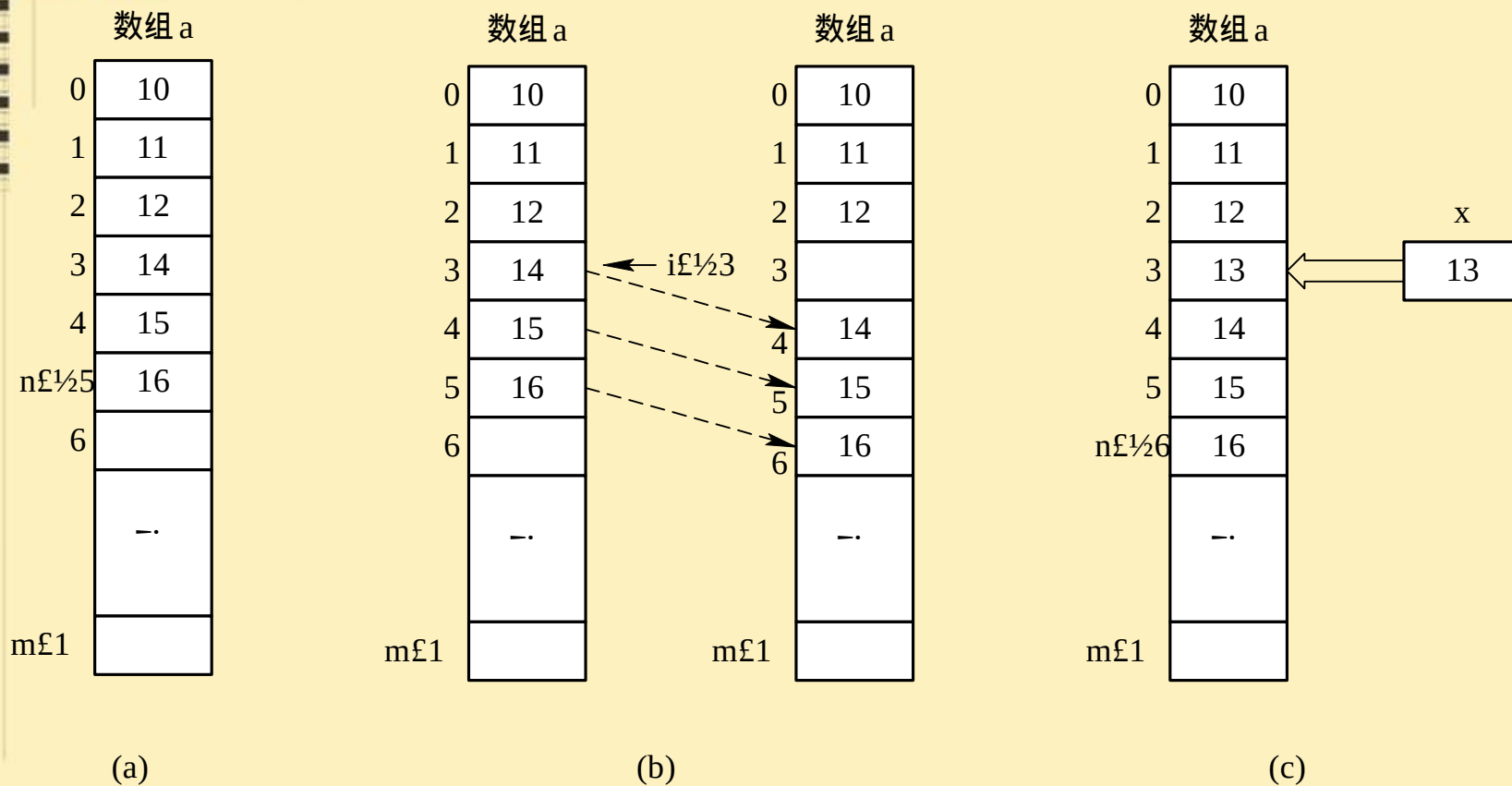


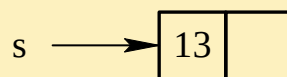
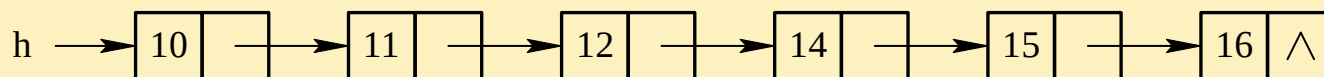
图 数组存储下线性表的插入过程

(a) 插入前；(b) 后移；(c) 插入

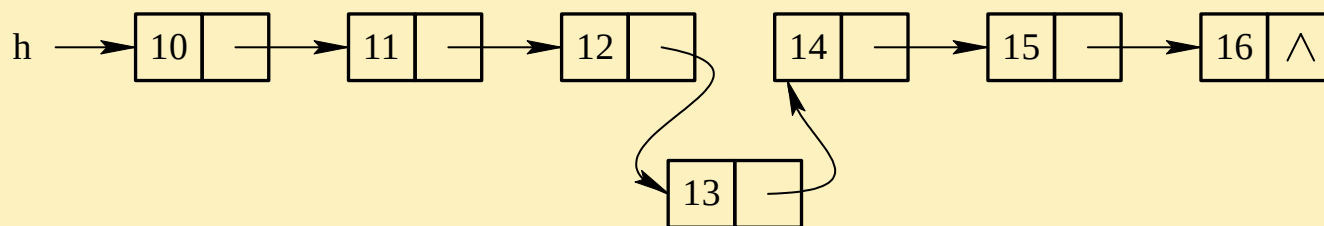
## 1.4.3 链表——插入



(a)



(b)



(c)

图 链式存储下线性表的插入过程

(a) 插入前；(b) 新结点赋值；(c) 链入

## 1.4.3 链表——删除

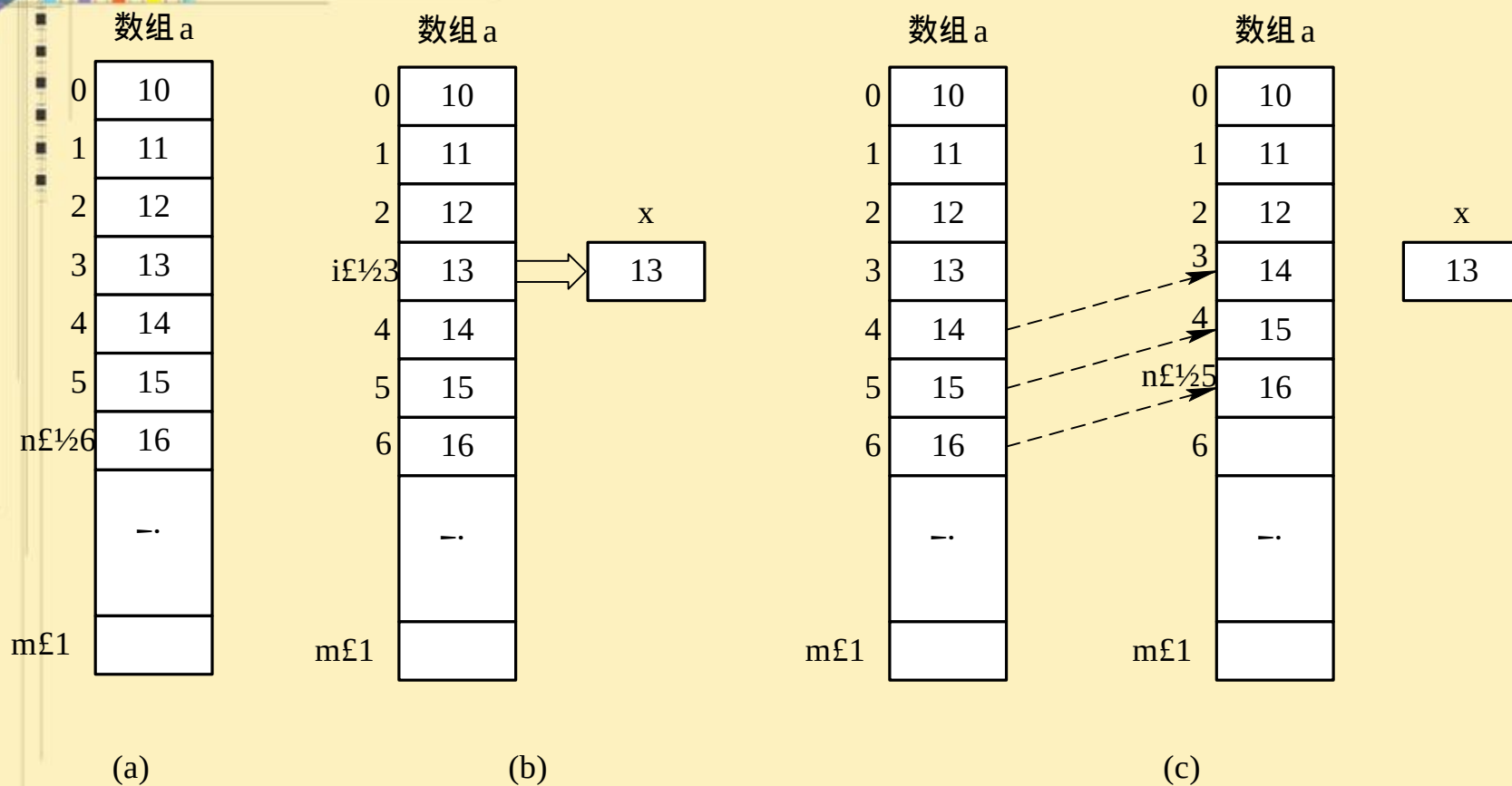
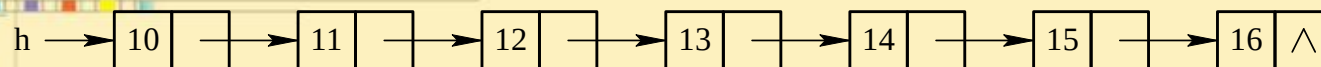


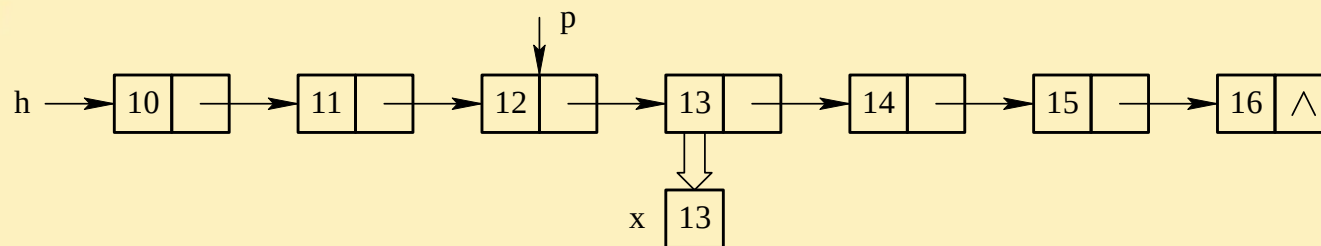
图 数组存储下线性表的删除过程

(a) 删除前；(b) 保存；(c) 前移

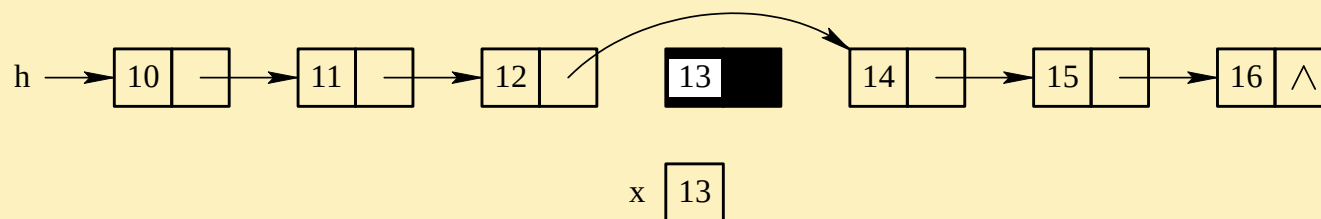
## 1.4.3 链表——删除



(a)



(b)



(c)

图 链式存储下线性表的删除过程

(a) 删除前；(b) 保存；(c) 删除

## 1.4.4 堆栈和队列

- 堆栈是一种只允许在称为栈顶的一端进行插入和删除运算的链表，也可以在数组中实现这些运算。
- 两种基本操作：
  - 如果  $S$  是一个堆栈，如果  $x$  是与  $S$  中元素类型相同的一个元素
  - **Push 运算** :  $\text{push}(S, x)$  把  $x$  加到  $S$  中，并改变堆栈的顶部，使它指向  $x$ 。
  - **Pop 运算** : 返回栈顶元素，并将它从堆栈中永久地移去。

## 1.4.4 堆栈和队列

- 堆栈也简称为**栈**，是限定在表的一端进行插入或删除操作的线性表。
- 进行插入或删除操作的一段称为**栈顶**（**top**），另一端称为**栈底**（**bottom**）。
- 插入元素又称为**入栈**（**push**），删除元素操作称为**出栈**（**pop**）。
- 不含元素的栈称为**空栈**。
- 堆栈元素的插入和删除只在栈顶进行，总是后进去的元素先出来，所以堆栈又称为**后进先出线性表**或**LIFO**（**Last-In-First-Out**）表。

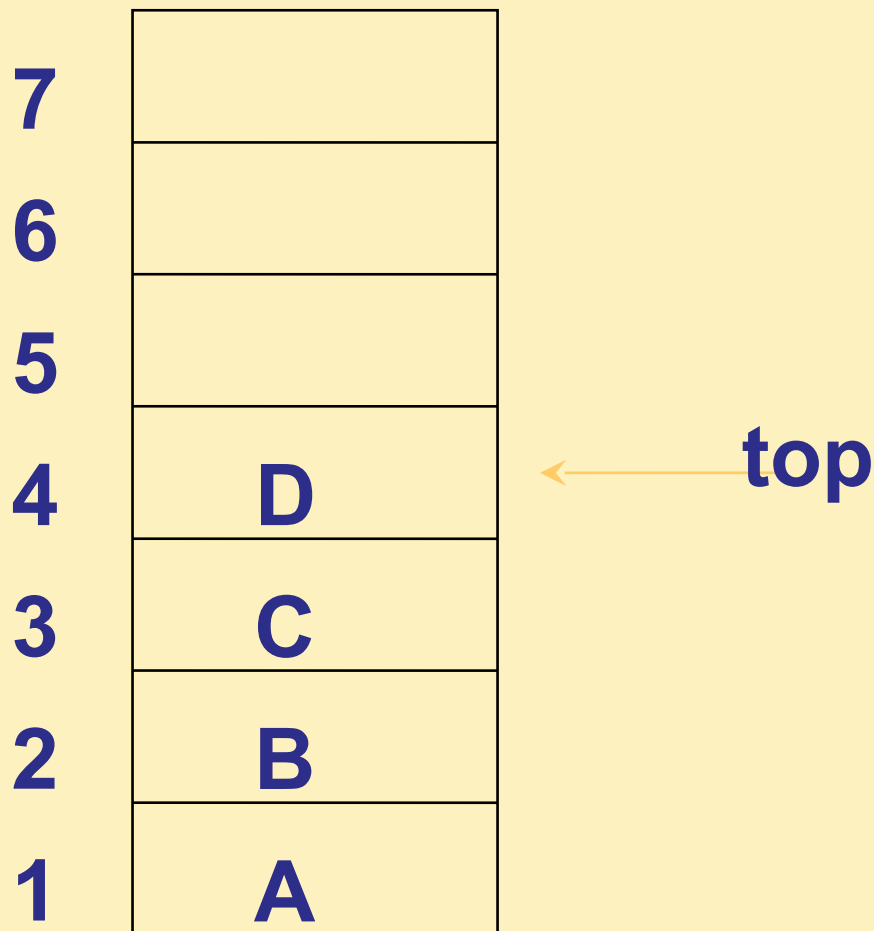


## 1.4.4 堆栈和队列

- 堆栈的最简单的表示方法是采用一维数组，为形象起见，一般在图中将堆栈画成竖直的。
- 设数组名为 STACK，其下标的下界为 1，上界为  $n$ 。
- 一般需用一个变量 top 记录当前栈顶的下标值，**top 也叫做栈指针。**

## 1.4.4 堆栈和队列

- 本例中  $\text{top}=4$



**STACK**

## 1.4.4 堆栈和队列

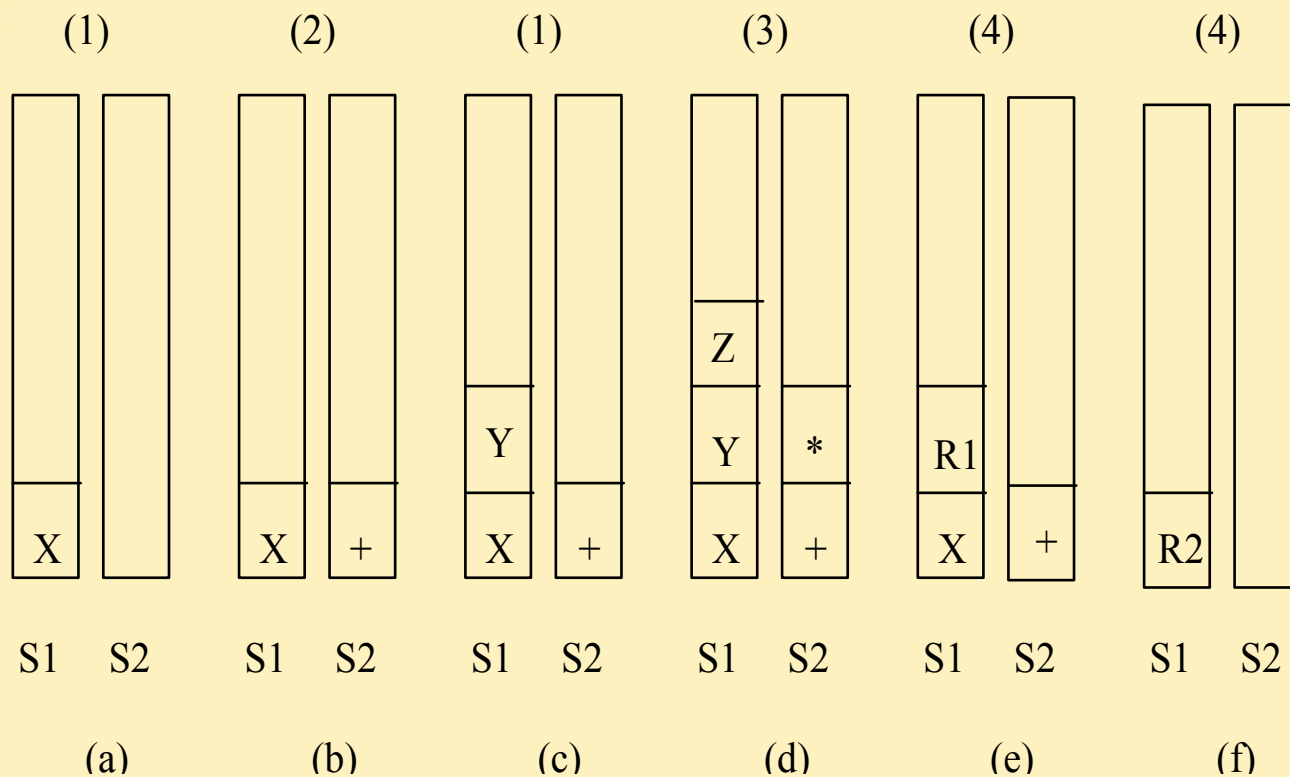
- 下面用一个简单的例子说明编译系统在处理算术表达式时，是如何应用堆栈这种数据结构的。
- 假定表达式的运算数都是使用单个字母表示的，式中无括号且只有加、减、乘、除4种运算，而没有更复杂的运算，例如表达式  $X+Y*Z$ 。

## 1.4.4 堆栈和队列

- 使用 S1 和 S2 两个堆栈，S1 用于存储运算数，S2 用于存储运算符。
- 编译系统处理时，将表达式从左向右逐个扫视一遍，并根据不同情况按以下原则处理：
  - 1) 若是运算数，则将其压入 S1 栈；
  - 2) 若是运算符且 S2 栈是空栈则将其压入 S2 栈；
  - 3) 若是运算符且 S2 栈为非空栈，且此运算符的级别高于 S2 栈顶运算符的级别，则将此运算符压入 S2 栈；
  - 4) 凡不属于上面三条的情况，则将 S2 的栈顶运算符与 S1 栈最上面的两个运算数出栈进行运算，并将运算结果压入 S1 栈。

## 1.4.4 堆栈和队列

- 图中每一步上面括号中的数字表示该步是按哪一条原则处理的。



## 1.4.4 堆栈和队列

- 队列是这样的一种链表：仅允许在称为队列尾部的链表一端进行插入运算，而只允许在称为队列头部的另一端进行所有的删除运算。
- 和堆栈一样，队列也可以在数组中实现这些运算。除了 push 运算把一个元素加入了队列尾部之外，队列支持的其他运算和堆栈一样。

## 1.4.4 堆栈和队列

- 队列是一种运算受限制的线性表，元素的添加在表的一端进行，而元素的删除在表的另一端进行。
- 允许添加元素的一端称为**队尾（Rear）**；允许删除元素的一端称为**队头（Front）**。
- 向队列添加元素称为**入队**，从队列中删除元素称为**出队**。
- 新入队的元素只能添加在队尾，出队的元素只能是删除队头的元素，队列的特点是先进入队列的元素先出队，所以队列也称作**先进先出表**或 FIFO（First-In-First-Out）表。



- ☐ 1  
☐ 2  
☐ 3  
☐ 4



## 1.4.4 堆栈和队列

- 存在的问题：

由于队列的入队操作是在两端进行的，随着元素的不断插入，删除，两端都向后移动，队列会很快移动到数组末端造成溢出，而前面的单元无法利用。

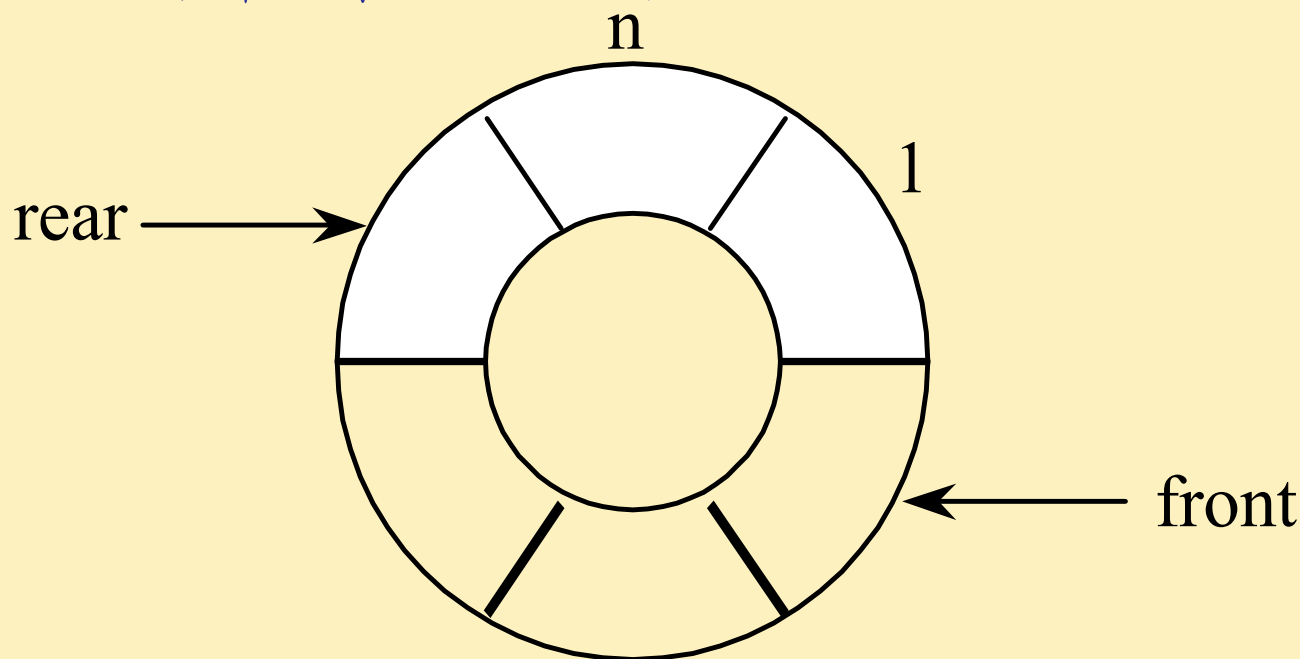
- 解决办法：

- 1) 每次删除一个元素后，将整个队列向前移动一个单元，保持队列头总固定在数组的第一个单元。
- 2) 将所用的数组想象成是头尾相接的圆环，当队列的尾端到达数组的末端（第  $n$  个单元）时，如果再插入元素可继续使队列向数组的前端（第 1 个单元）延长，此队列称为循环队列。

## 1.4.4 堆栈和队列

循环队列:

- 图中阴影部分为队列中元素。
- 如何判断一个循环队列是满还是空?



## 1.4.4 堆栈和队列

如何判断队列是满还是空？

- **满**：队尾经过一个循环而到达队首的前一个单元时，这种情况下如果再插入新的元素时，新元素就要把原队头的元素覆盖，因此，当  $r=f$  时，插入新的元素会造成队列首尾重叠；
- **空**：在队列进行删除运算时，当  $f=r$  时表明删除的是队列的最后一个元素，删除这个元素后，队列就变成空队列。

## 1.4.5 图

- 图是一种比线性表和树更为复杂的数据结构。在图中，元素间的关系是多对多的，即任何两个元素都有可能存在关系。
- 图的应用非常广泛，已渗入到诸如语言学、逻辑学、物理、化学、电讯工程、计算机科学以及数学的其它分支中（日常生活中的交通图等）。
- 在离散数学中，图论是专门研究图的性质的数学分支。
- 在数据结构中对图的讨论主要侧重于图在计算机中的存储方式和有关操作的算法。

## 1.4.5 图

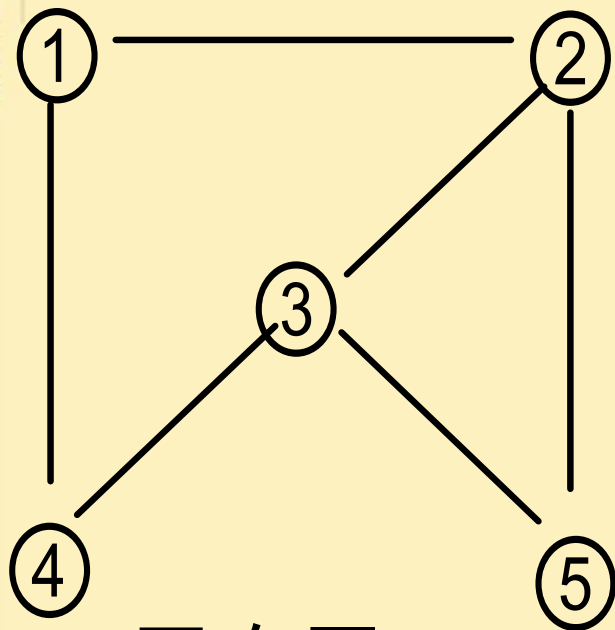
- 图：

- 图  $G=(V,E)$  由一个顶点集合  $V=\{v_1, v_2, \dots, v_n\}$  和一个边的集合  $E$  组成。  $G$  可以是有向的也可以是无向的。如果  $G$  是**无向的**，那么  $E$  中的每一条边都是无序的顶点对；如果  $G$  是**有向的**，那么  $E$  中的每一条边都是一个有序的顶点对。

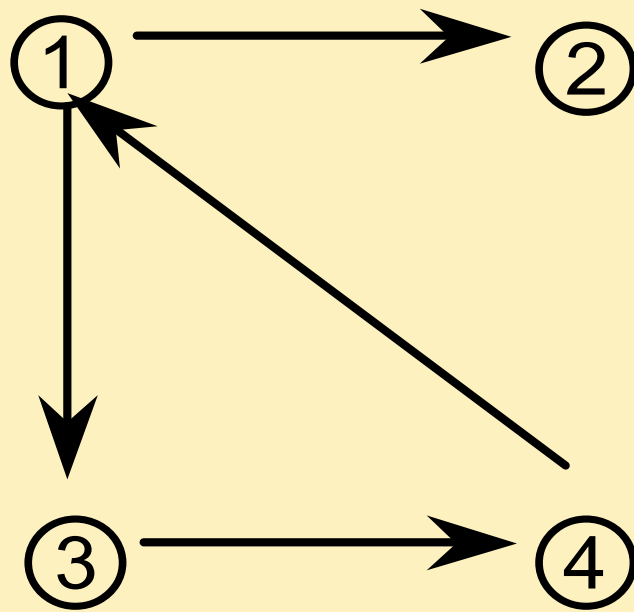
- 邻接：

- 设  $(v_i, v_j)$  是  $E$  的一条边，如果图是无向的，那么  $v_i$  和  $v_j$  互相邻接，如果图是有向的，那么  $v_j$  邻接于  $v_i$ ，但  $v_i$  不邻接于  $v_j$ ，除非  $(v_j, v_i)$  也是  $E$  中一条边。

## 1.4.5 图



无向图  $G_1$



有向图  $G_2$



## 1.4.5 图

- 度：
  - 无向图中顶点的度是和它邻接的顶点的个数，有向图中顶点  $v_i$  的入度和出度分别是连接到  $v_i$  的边和从  $v_i$  连接到其他顶点的边的数目。
- 路径：
  - 图中从顶点  $v_1$  到  $v_k$  的路径是一个顶点的序列  $v_1, v_2, \dots, v_k$ ，其中的  $(v_i, v_{i+1}), 1 \leq i \leq k-1$ ，是图的一条边。
  - 路径的长度是路径中边的数目，因此，路径  $v_1, v_2, \dots, v_k$  的长度是  $k-1$ 。如果所有的顶点都不同，那么路径是简单的；如果  $v_1 = v_k$ ，那么路径是一个回路。
  - 奇数长度的回路，其边的数目是奇数，类似的方法可以定义偶数长度的回路。
  - 没有回路的图称为无回路图。如果有一条路径从顶点  $u$  开始到顶点  $v$  结束，那么称顶点  $v$  从  $u$  出发是可达的。

## 1.4.5 图

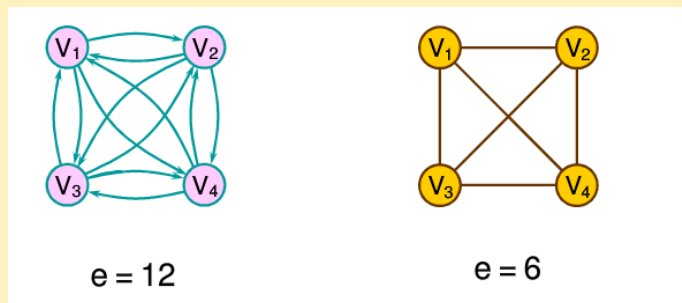
- 连通的：

- 在无向图中，如果每个顶点从其他每个顶点出发都是可达的，那么这个无向图是连通的，否则就是不连通的。一个图的连通分支是图的最大连通子图。
- 在有向图中，如果子图中每一对顶点  $u$  和  $v$  满足  $v$  是从  $u$  出发可达的，同时， $u$  也是从  $v$  出发可达的，这个子图就称为强连通分支。

## 1.4.5 图

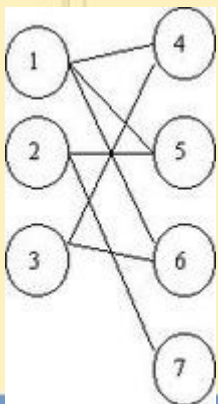
- 完全图：

- 如果一个无向图的每一对顶点之间都恰有一条边，那么这个无向图就称为完全图。如果一个有向图的每个顶点到所有其他顶点之间都恰有一条边，那么这个有向图称为完全图。



- 二分图：

- 如果在无向图  $G=(V,E)$  中， $V$  可以分成两个不相交的子集  $X$  和  $Y$ ，使得  $E$  中的每一条边的一端在  $X$  中而另一端在  $Y$  中，则无向图称为二分图。如果在任意的顶点  $x \in X$  和  $y \in Y$  之间都有一条边，它就称为完全二分图。



## 1.4.5 图——表示方法

- 邻接矩阵：

- 图  $G=(V,E)$  可以用一个布尔矩阵  $M$  方便的表示，它是这样定义的，当且仅当  $(v_i, v_j)$  是  $G$  中的一条边时， $M[i,j]=1$ ，矩阵  $M$  称为  $G$  的邻接矩阵。有  $n$  个顶点的图的邻接矩阵有  $n^2$  项。

- 邻接表：

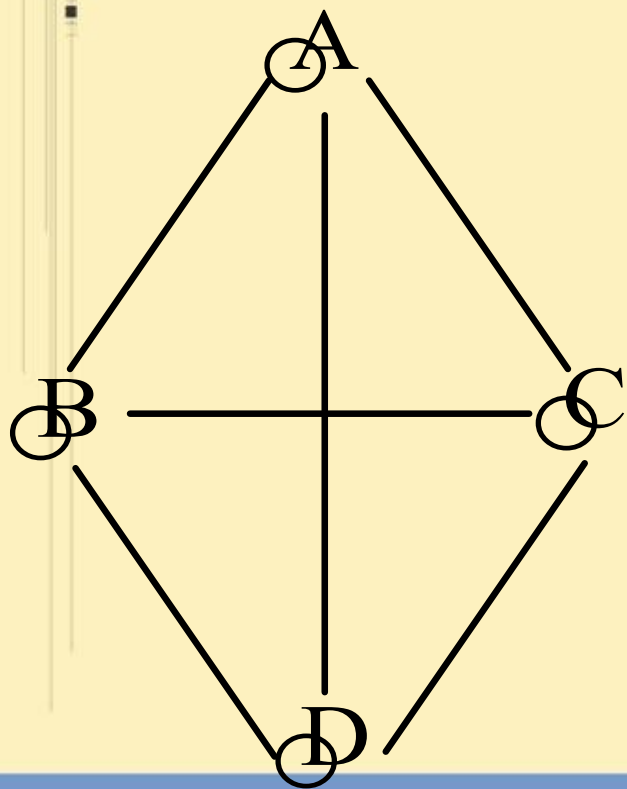
- 图的另一种表示法是邻接表表示法。一个顶点的所有邻接顶点用一个链表来表示，共有  $|V|$  个这样的表。邻接表要花费  $O(m+n)$  空间来表示有  $n$  个顶点和  $m$  条边的图。

## 1.4.5 图——邻接矩阵

- **邻接矩阵**：表示顶点之间相邻关系的矩阵。所谓两顶点的相邻关系即它们之间有边相连。
- **邻接矩阵**是一个  $(n \times n)$  阶方阵， $n$  为图的顶点数，它的每一行分别对应图的各个顶点，它的每一列也分别对应图的各个顶点。我们规定矩阵的元素为：

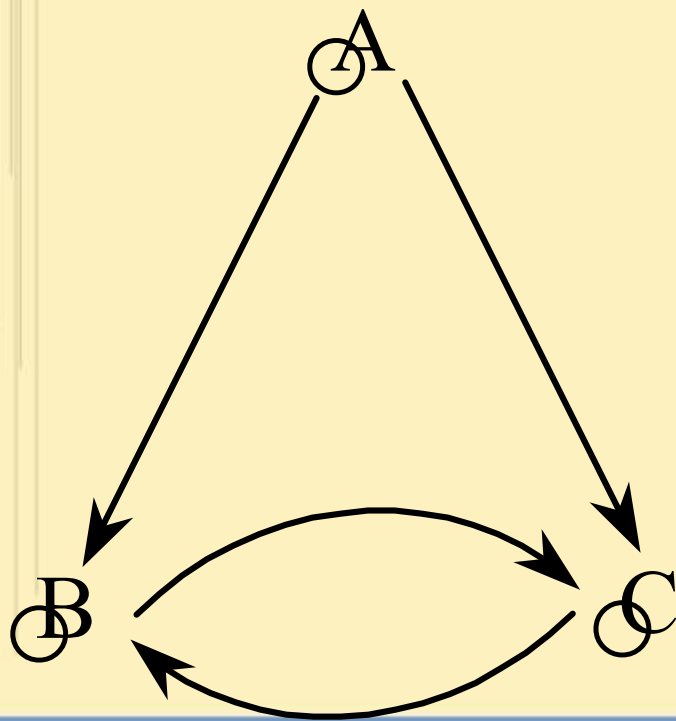
$$A[i, j] = \begin{cases} 1 & \text{对无向图若存在}(V_i, V_j)\text{边，对有向图若存在} \langle V_i, V_j \rangle \text{边} \\ 0 & \text{反之} \end{cases}$$

## 1.4.5 图——无向图的邻接矩阵



$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \begin{matrix} A \\ B \\ C \\ D \end{matrix}$$

## 1.4.5 图——有向图的邻接矩阵



$$B = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{matrix} A \\ B \\ C \end{matrix}$$



## 1.4.5 图——邻接矩阵的若干说明

- 无向图的邻接矩阵是**对称的**，如果  $A[i, j]=1$ ，必有  $A[j, i]=1$ 。这说明，只输入和存储其上三角阵元素即可得到整个邻接矩阵。
- 一般有向图的邻接矩阵是不对称的， $A[i, j]$  不一定等于  $A[j, i]$ 。
- 邻接矩阵用二维数组即可存储，定义如下：  
$$\text{int adjmatrix} = \text{ARRAY}[n][n];$$
- 如果图的各边是带权的，只需将矩阵中的各个 1 元素换成相应边的权即可。

## 1.4.5 图——邻接表

- 邻接表是图的一种链式存储结构。
- 在邻接表结构中，对图中每个顶点建立一个单链表，第  $i$  个单链表中的结点表示依附于该顶点  $V_i$  的边，即对于无向图每个结点表示与该顶点相邻接的一个顶点；对于有向图则表示以该顶点为起点的一条边的终点。
- 一个图的邻接矩阵表示是唯一的，但其邻接表表示是不唯一的。因为在邻接表的每个单链表中，各结点的顺序是任意的。

|         |                 |
|---------|-----------------|
| 顶点信息    | 指向链表中的<br>第一个结点 |
| verdata | firarc          |

头结点

```
typedef struct tnode
{ int vexdata;
 struct tnode* firarc;
}TD;
```

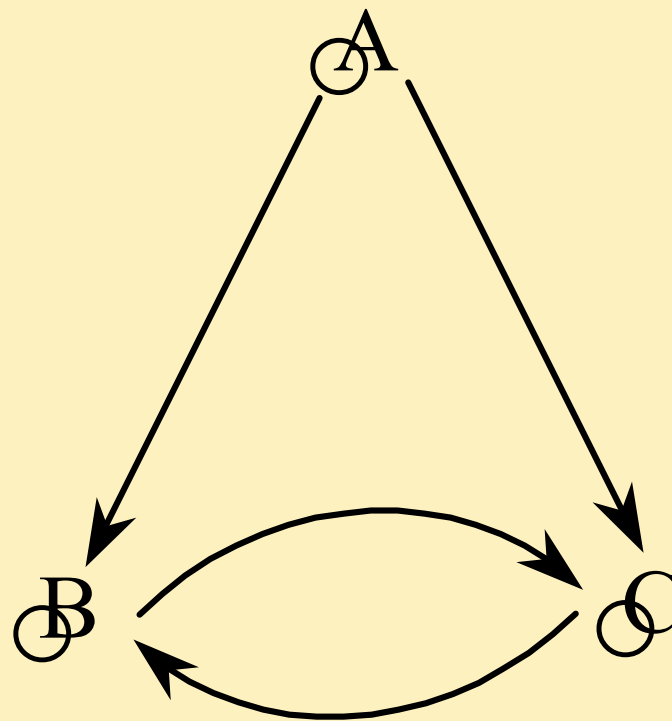
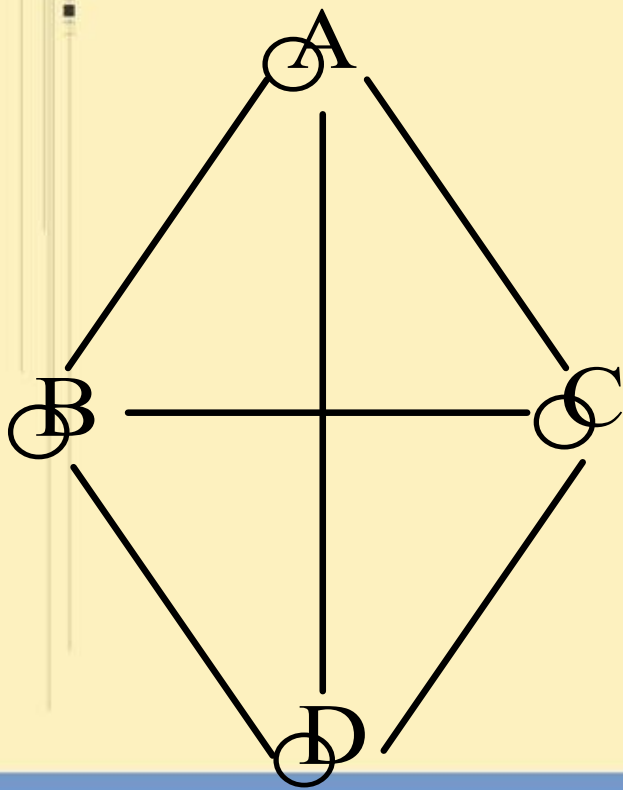
TD ha[M];

|            |                  |              |
|------------|------------------|--------------|
| 顶点的邻<br>接点 | 边或弧相关的<br>信息（权值） | 指向下一<br>条边或弧 |
| adjvex     | data             | nextarc      |

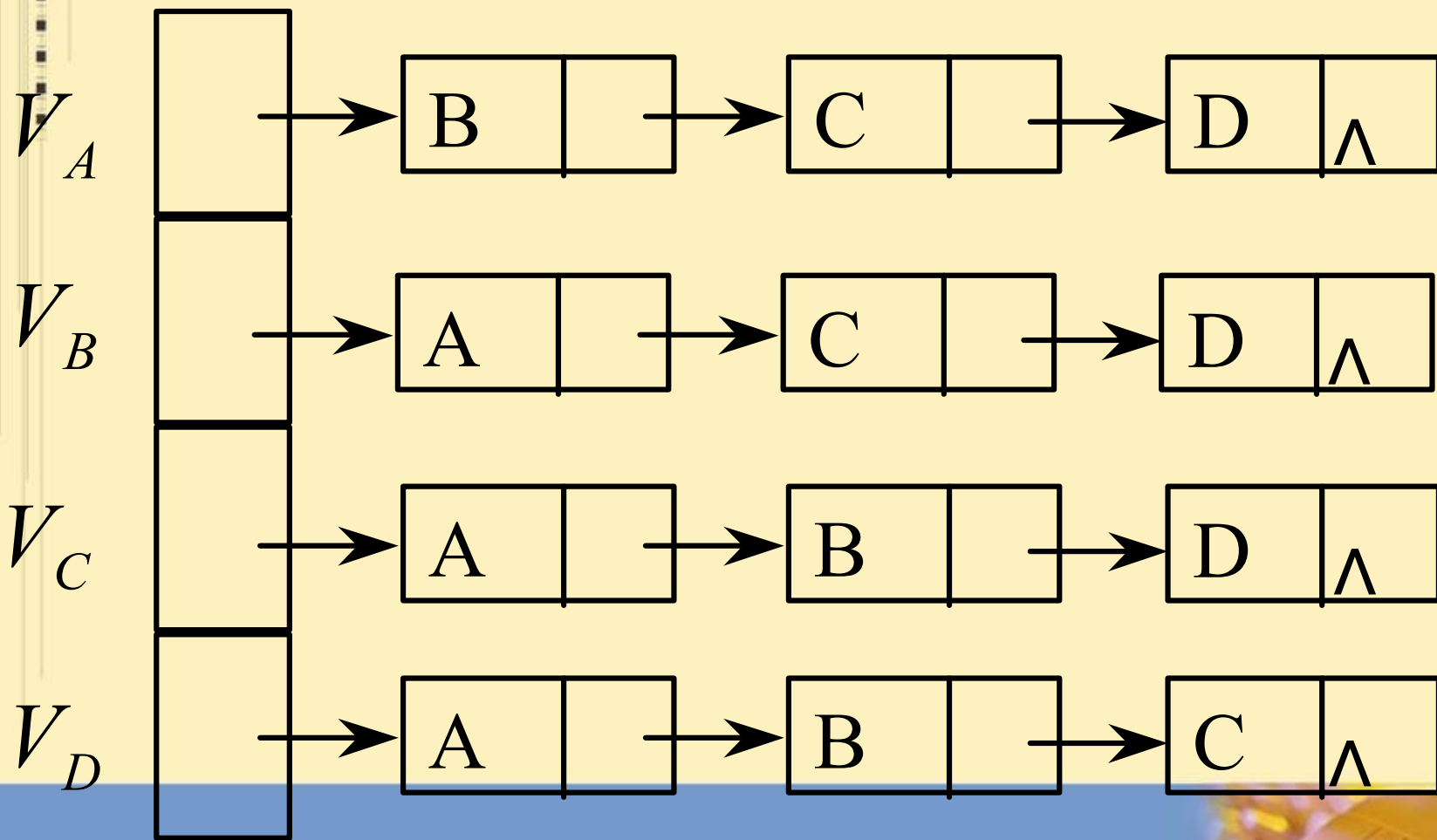
边结点

```
typedef struct node
{ int adjvex,data;
 struct node* nextarc;
}JD;
```

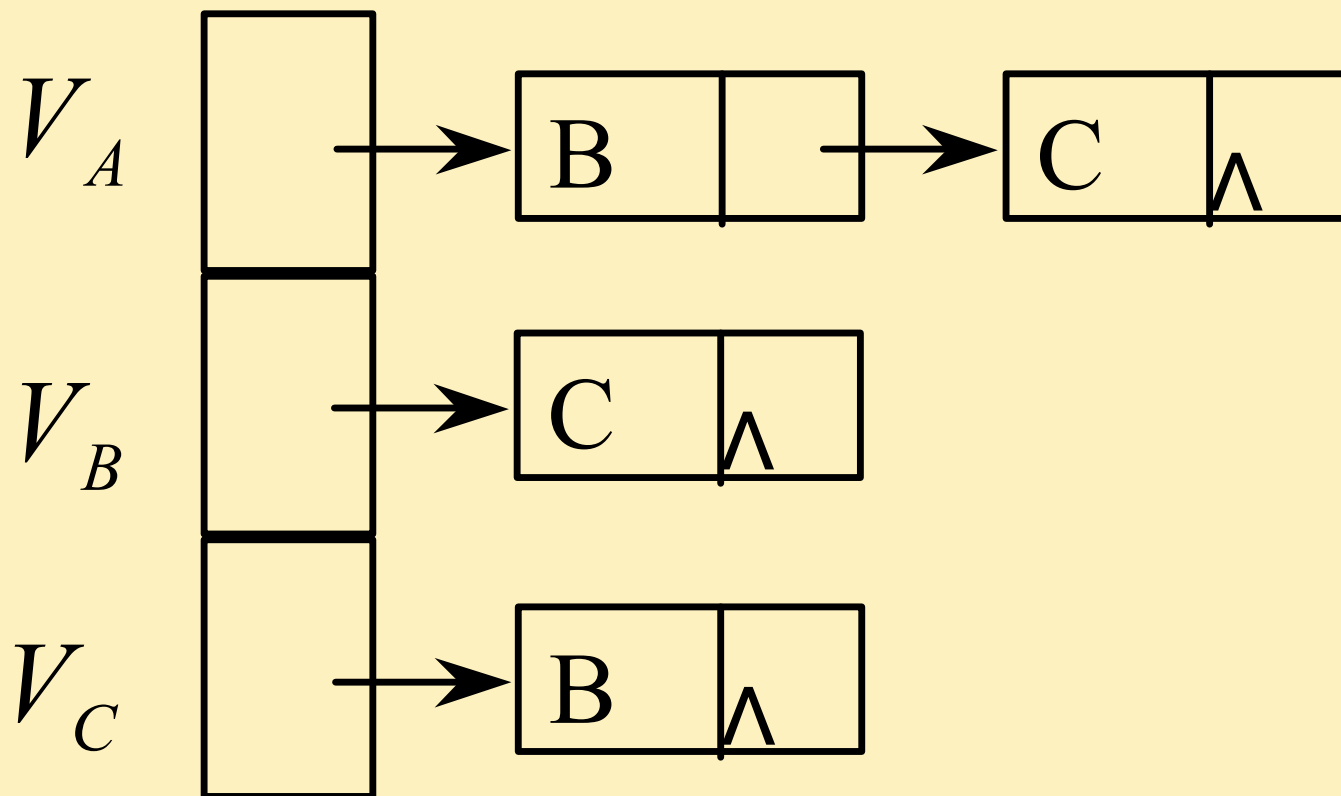
## 1.4.5 图——无向图、有向图



## 1.4.5 图——无向图的邻接表



## 1.4.5 图——有向图的邻接表

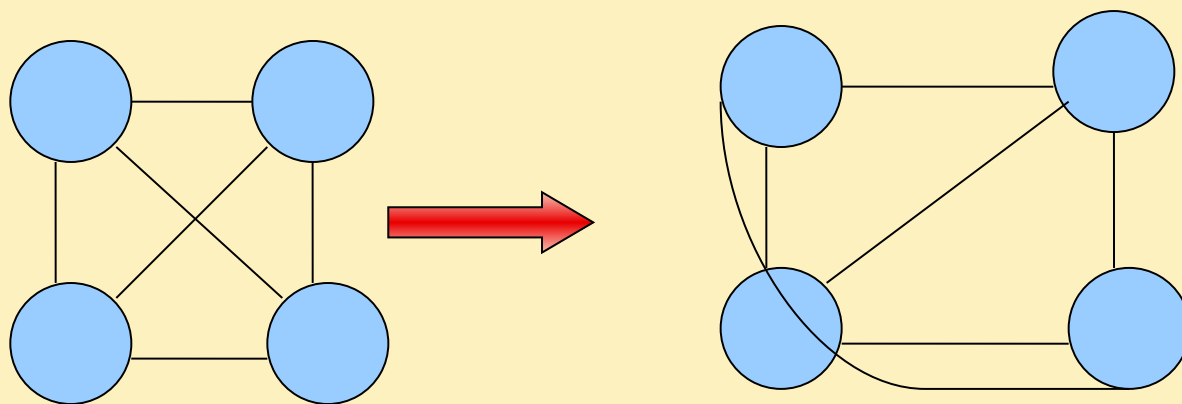


## 1.4.5 图——邻接表的若干说明

- 对于无向图的邻接表来说，一条边对应两个单链表结点，邻接表结点总数是边数的 2 倍。
- 在无向图的邻接表中，各顶点对应的单链表的结点数（不算表头结点）就等于该顶点的度数。
- 在有向图邻接表中，一条弧对应一个表结点，表结点的数目和弧的数目相同。
- 在有向图邻接表中，单链表的结点数就等于相应顶点的出度数。
- 要求有向图中某顶点的入度数，需扫视邻接表的所有单链表，统计与顶点标号相应的结点个数。

## 1.4.5 图——平面图

- 如果图  $G=(V,E)$  可以嵌入平面而没有任何边互相穿越，这个图就是平面图。





## 1.4.5 图——重要关系

令  $n, m, r$  分别表示任意一个平面图中的顶点数、边数和区域数，这三个参数之间的关系有欧拉公式：

$$n - m + r = 2, \quad \text{或} \quad m = n + r - 2$$

而且还有  $m \leq 3n - 6$ .

等号在图是三角形化时成立，即图的每一个区域（包括无界区域）都是三角形的。

上面的关系蕴含着在任何平面图中  $m = O(n)$ ，这样存储一个平面图所需要的空间量仅仅是  $O(n)$ 。

## 1.4.6 树、根树、二叉树

- 一个自由树（或简称树）是**不包含回路**的连通无向图。一个森林是顶点不相交的树的集合，即它们没有公共的顶点。

- 定理 3.1



如果  $T$  是有  $n$  个顶点的树，那么

- $T$  中任意两个顶点有唯一的一条路径连通；
- $T$  恰好有  $n-1$  条边；
- 在  $T$  中加上一条边将构成一个回路。

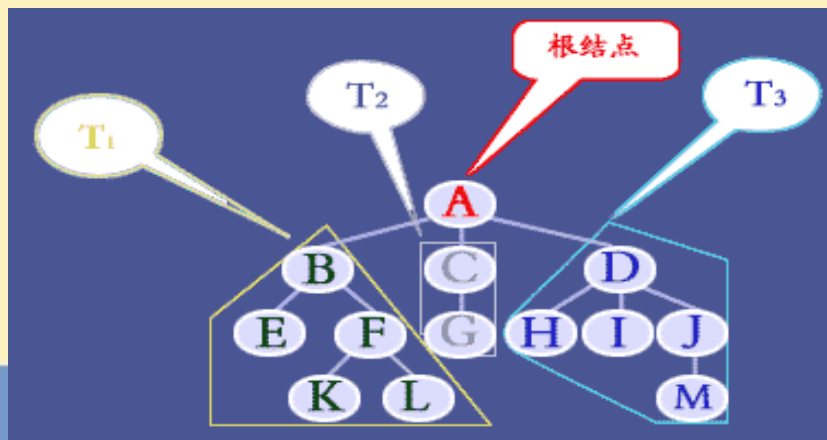


## 1.4.6 树、根树、二叉树

- 树是一类重要的非线性结构。
- 树是结点之间有分支，并且具有层次关系的结构，它非常类似于自然界中的树。
- 树在客观世界中是大量存在的，例如家谱、行政组织机构都可用树形象地表示。
- 树在计算机领域中也有着广泛的应用，例如
  - 在编译程序中，用树来表示源程序的语法结构；
  - 在数据库系统中，可用树来组织信息；
  - 在分析算法的行为时，可用树来描述其执行过程。等等。

## 1.4.6 树、根树、二叉树

- 根树  $T$  是一棵带有特殊顶点  $r$  的树，顶点  $r$  称为  $T$  的**根**。
- 在  $T$  中，如果顶点  $v_i$  在从根到  $v_j$  的路径上，并且和  $v_j$  相邻，就说  $v_i$  是  $v_j$  的**父顶点**，同时， $v_j$  是  $v_i$  的**子顶点**。
- 一个顶点的子顶点之间彼此是**兄弟**。没有子顶点的顶点是树根的**叶子**，其他所有的顶点都称为**内部顶点**。
- 在从根顶点到顶点  $v$  的路径上的顶点  $u$  是  $v$  的**祖先**，如果  $u \neq v$ ,  $u$  就是  $v$  的**真祖先**；在从顶点  $v$  到叶子的路径上的顶点  $w$  是  $v$  的**后代**，如果  $w \neq v$ ,  $w$  就是  $v$  的**真后代**。



## 1.4.6 树、根树、二叉树

- 子树：

- 从顶点  $v$  为根的子树是包括顶点  $v$  和它的真后代的树

- 深度：

- 在根树中顶点  $v$  的深度是从根顶点到  $v$  的路径的长度。这样，根顶点的深度为 0。

- 高度：

- 顶点  $v$  的高度定义为从顶点  $v$  到叶子中最长路径的长度，一棵树的高度是根顶点的高度。

## 1.4.6 树、根树、二叉树

### 根数遍历：

有几种方法可以对一棵根树的顶点进行有系统的遍历和排序，其中最重要的三种是前序，中序和后序遍历。令  $T$  是一颗树，它的根顶点是  $r$ ， $T_1, T_2, \dots, T_n$  是子树。

- 前序遍历：

在  $T$  顶点的前序遍历中，先访问根顶点  $r$ ，然后以前序遍历  $T_1$  的顶点，之后前序遍历  $T_2$  的顶点。如此继续，直到前序遍历  $T_n$  的所有顶点。

- 中序遍历：

在  $T$  顶点的中序遍历中，先以中序遍历  $T_1$  的顶点，然后访问根顶点  $r$ ，之后中序遍历  $T_2$  的顶点。如此继续，直到中序遍历  $T_n$  的所有顶点。

- 后序遍历：

在  $T$  顶点的后序遍历中，先以后序遍历  $T_1$  的顶点，然后后序遍历  $T_2$  的顶点。如此继续，直到后序遍历  $T_n$  的所有顶点，最后访问根顶点  $r$ 。



100

-



## 1.4.6 树、根树、二叉树

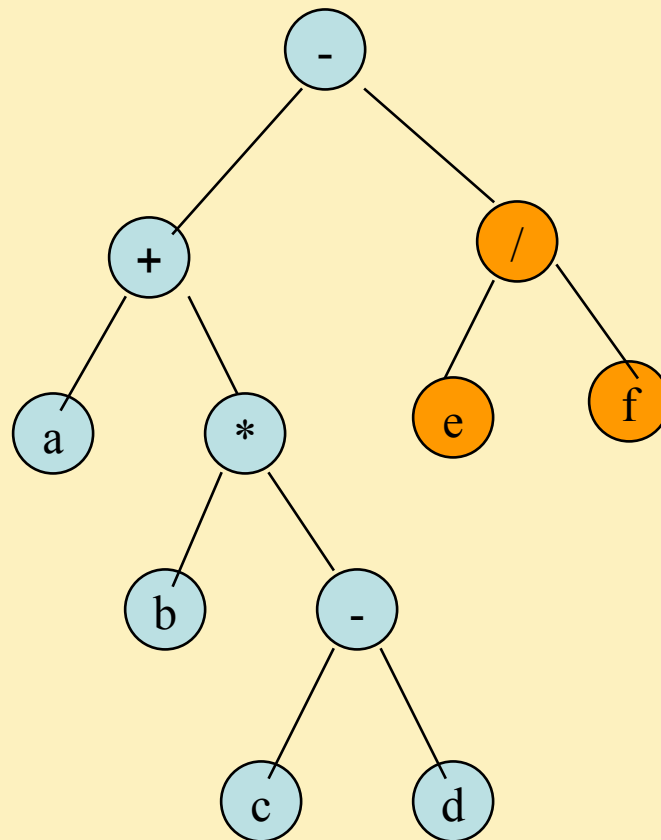
根数遍历：

例：如图所示的二叉树表达式  
(a+b\*(c-d)-e/f)

•若先序遍历，其先序序列为：  
-+a\*b-cd/ef

•按中序遍历，其中序序列为：  
a+b\*c-d-e/f

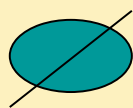
•按后序遍历，其后序序列为：  
abcd-\*+ef/-



## 1.4.6 树、根树、二叉树

### 定义

- 二叉树是节点的一个有限集合，集合或者为空，或者由一个根节点  $r$  和称为**左右子树**的两个不相交的二叉树组成。这些子树的根称为  $r$  的左右儿子。
- 二叉树与根树之间有两个很重要的不同点：
  - 第一，二叉树可以为空而根树不能为空。
  - 第二，由于二叉树有左右子树的区别，使得图 .3.6 中 (a)、(b) 两颗二叉树是不同的，但是如果是根树，他们就不能区分。



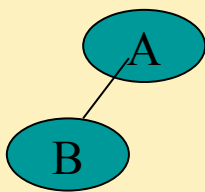
(a)

空二叉树



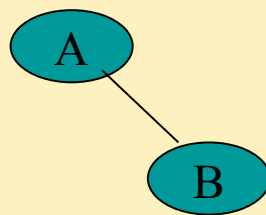
(b)

根和空的左右子树



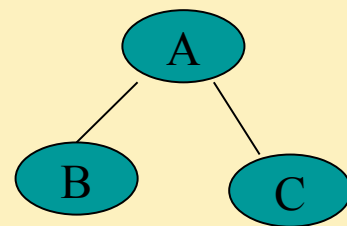
(c)

根和左子树



(d)

根和右子树

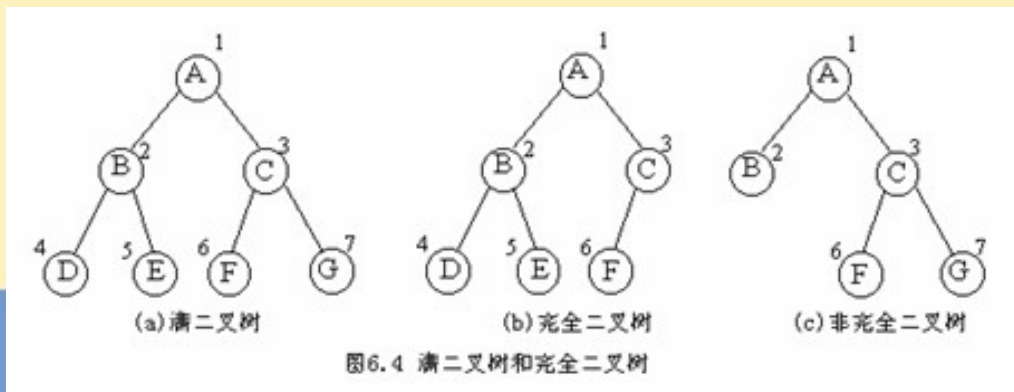


(e)

根和左右子树

## 1.4.6 树、根树、二叉树

- 满的和完全的：
  - 如果二叉树中每个内部节点都正好有两个儿子，这样的二叉树称为**满的**；如果二叉树是满二叉树，而且所有的叶子有同样的深度（如在同一层），那么这种二叉树称为**完全二叉树**。
- 几乎完全的：
  - 如果一颗二叉树除了最右边位置上的一个或者几个叶子可能缺少外，它是丰满的，我们定义它为几乎完全的二叉树。



## 1.4.6 树、根树、二叉树

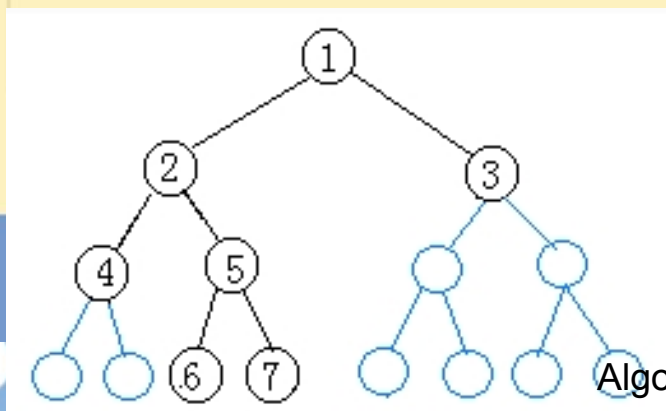
### 存储结构：

- 一颗有  $n$  个顶点的完全（或几乎完全）的二叉树可以用数组  $A[1..n]$  来有效的表示，数组根据下面的简单关系列出二叉树的顶点：
  - 如果存储在  $A[j]$  中的顶点有左儿子和右儿子，就把他们分别存储在  $A[2j]$  和  $A[2j+1]$  中。
  - 存储在  $A[j]$  中的顶点的父顶点存储在  $A[\lfloor j/2 \rfloor]$  中。

## 1.4.6 树、根树、二叉树

### 二叉树的顺序存储结构：

- 二叉树的顺序存储结构用一组连续的存储单元存储二叉树的数据元素。因此，必须把二叉树的所有结点安排成为一个恰当的序列，以使得结点在这个序列中的相互位置能反映出结点之间的逻辑关系，为此使用编号法。即从树根起，自上层至下层，每层自左至右的给所有结点编号。
- ```
#define MAX_TREE_SIZE 100  
typedef TElemType SqBiTree[MAX_TREE_SIZE];  
SqBiTree bt;
```



1	2	3	4	5	0	0	0	0	6	7	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

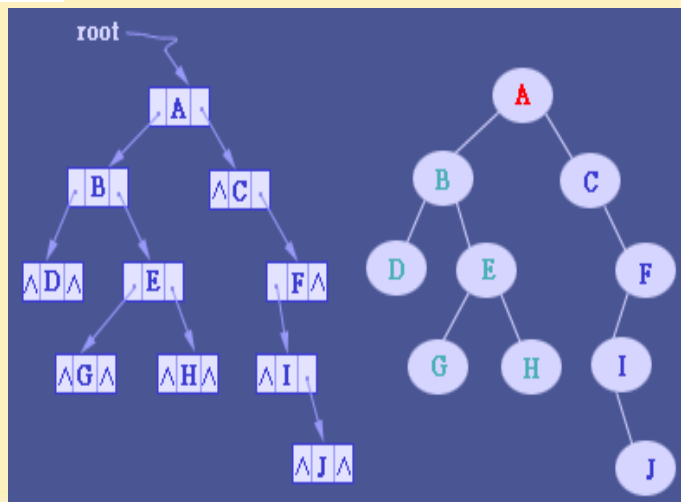
1.4.6 树、根树、二叉树

二叉树的链式存储结构：二叉链表

- 由二叉树的定义可知，二叉树的结点由一个数据元素和分别指向其左、右子树的两个分支构成，则表示二叉树的链表结点中至少包含三个域：数据域、左指针域及右指针域。

Lchild	item	Rchild
--------	------	--------

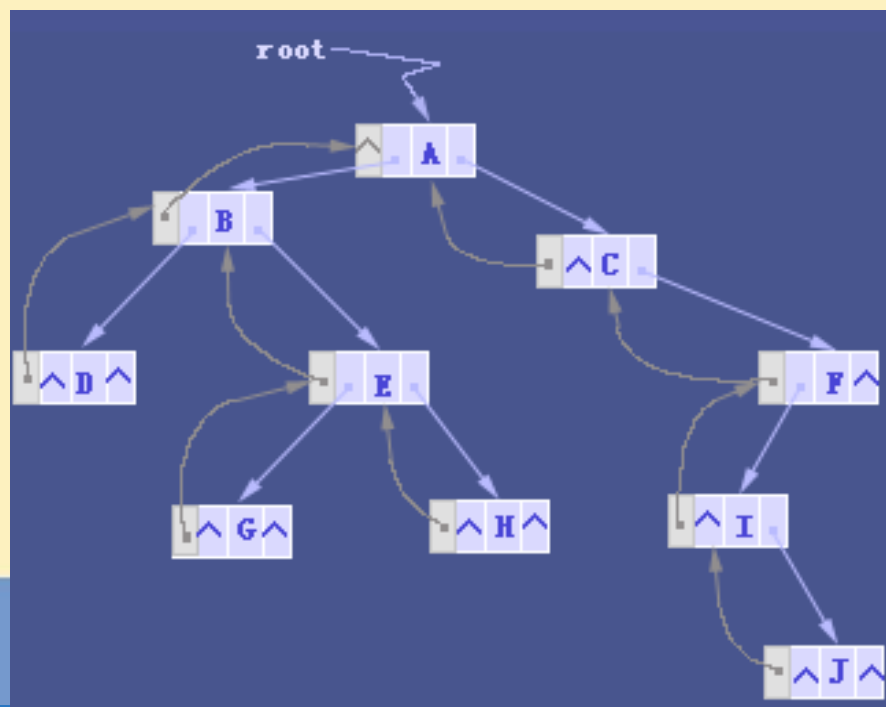
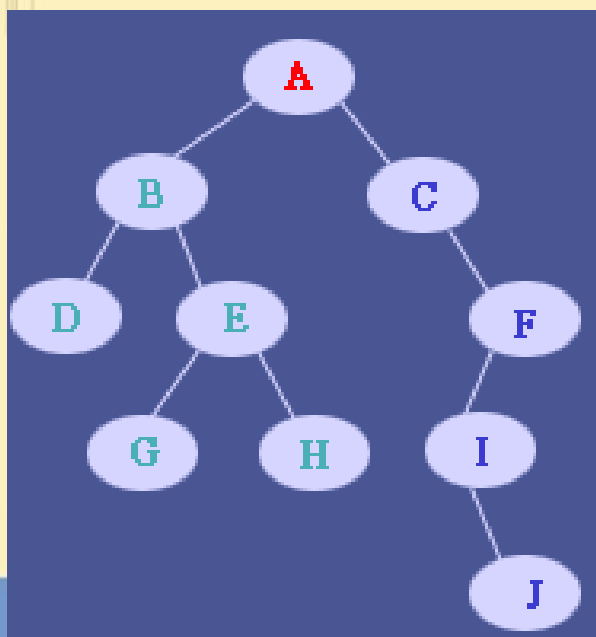
- typedef struct BiTNode{
✂ TElemType data;
✂ struct BiTNode *lchild,*rchild;
✂ }BiTNode,*BiTree;



1.4.6 树、根树、二叉树

- 二叉链表的存储特点是寻找孩子结点容易，双亲比较困难。因此，若需要频繁地寻找双亲，可以给每个结点添加一个指向双亲结点的指针域，其结点结构如下所示。

lchild	data	parent	rchild
--------	------	--------	--------



1.4.6 树、根树、二叉树

二叉树的一些定量特征

- 在下面的观察结论中，列出了二叉树的层、顶点数和高度间的一些有用的关系。
- 观察结论 3.1 在二叉树中，第 j 层的顶点数最多是 2^j 。
- 观察结论 3.2 令二叉树 T 的顶点数是 n ，高度是 k ，那么：

$$n \leq \sum_{j=0}^k 2^j = 2^{k+1} - 1.$$

如果 T 是完全的，等号成立。如果 T 是几乎完全的，那么有

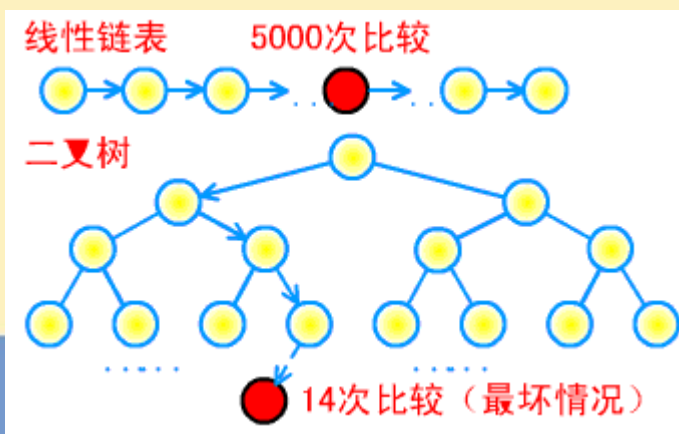
$$2^k \leq n \leq 2^{k+1} - 1.$$

其他观察结论见 P72

1.4.6 树、根树、二叉树

二叉搜索树

- 事实上，二分查找的过程，隐含地对应一棵二叉树形，但这样的二叉树形仅是逻辑上的，不能进行插入和删除操作。如果用二叉树形来组织表，则我们将会得到较好的动态查找算法。
- 一般来说，二叉树结构可以显著地改善搜索的性能，因为达到一个树的路径最长不超过树深度。



1.4.6 树、根树、二叉树

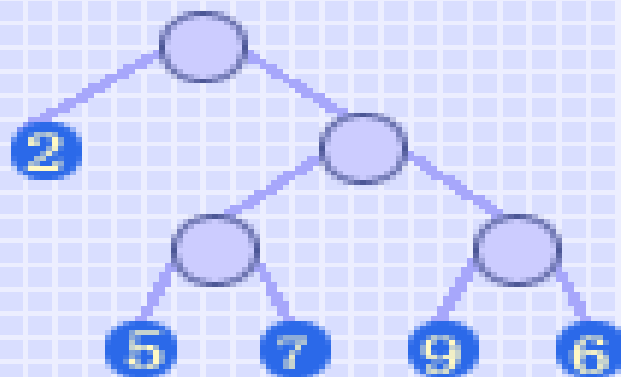
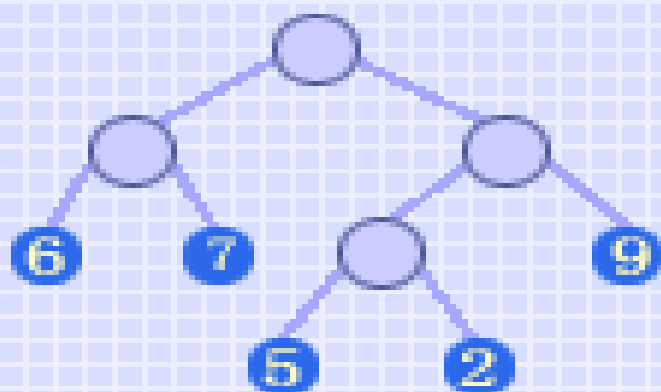
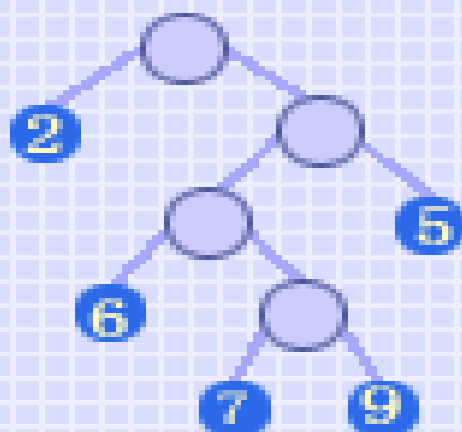
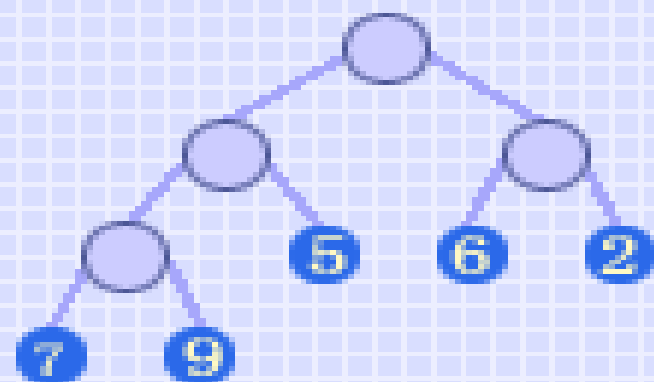
- 二叉搜索树是顶点用线性集中的元素来标记的一种二叉树，标记的方法是：所有存储在顶点 v 的左子树中的元素都小于存储在 v 中的元素，所有存储在顶点 v 中的右子树中的元素都大于存储在 v 中的元素。
- 用二叉搜索树来表示一个集合并不是唯一的，最坏的情况它可能是一棵退化的树，即这棵树的每一个内部顶点都恰好有一个儿子。
- 这种数据结构支持的操作有插入、删除、测试成员身份和检索最大或最小值。
- P72

1.4.7 赫夫曼树及其应用

- **赫夫曼树**：带权路径长度最短的树称为赫夫曼树或最优树。
- **路径**：树的根结点到其余结点的分支。
- **结点的路径长度**：从根结点到该结点的路径上分支的数目。
- **结点的带权路径长度**：从树根到该结点之间的路径长度与该结点上所带权值的乘积。
- **树的带权路径长度**：树中所有叶子结点的带权路径长度之和。

$$WPL = \sum_{k=1}^n w_k l_k$$

- 在所有含 n 个叶子结点、并带相同权值的 m 叉树中，必存在一棵其带权路径长度取最小值的树，称为“**最优树**”



$$WPL = 7 \times 3 + 9 \times 3 + 5 \times 2 + 6 \times 2 + 2 \times 2 = 74 \quad (\text{左上图})$$

$$WPL = 2 \times 1 + 6 \times 3 + 7 \times 4 + 9 \times 4 + 5 \times 2 = 94 \quad (\text{右上图})$$

$$WPL = 6 \times 2 + 7 \times 2 + 5 \times 3 + 2 \times 3 + 9 \times 2 = 65 \quad (\text{左下图})$$

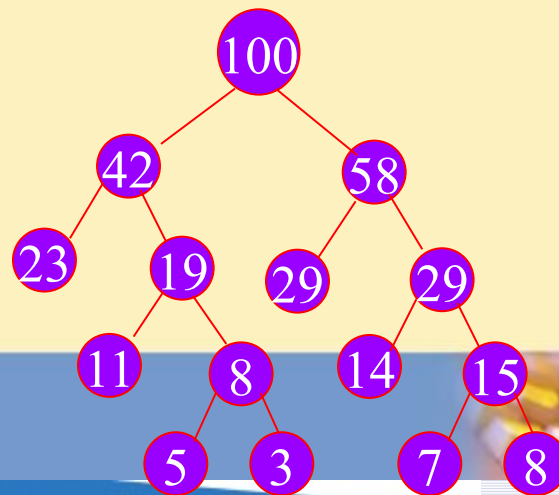
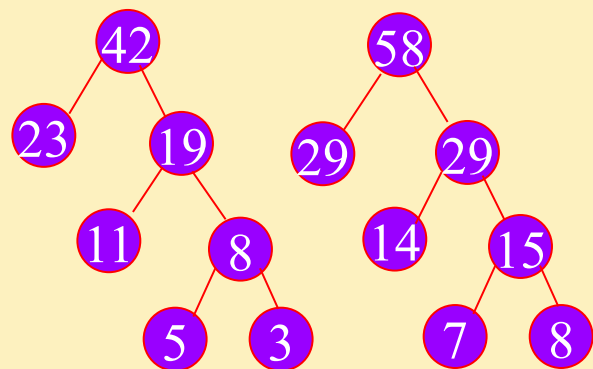
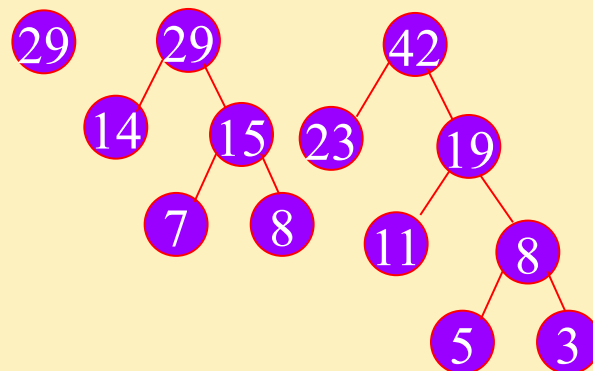
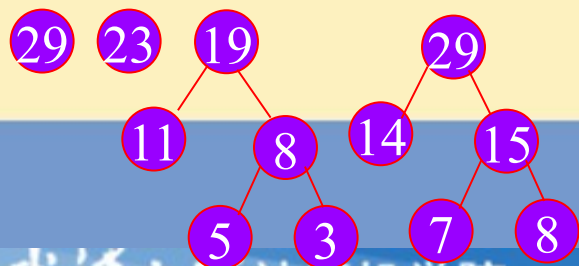
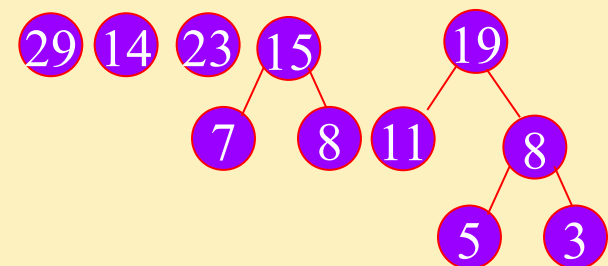
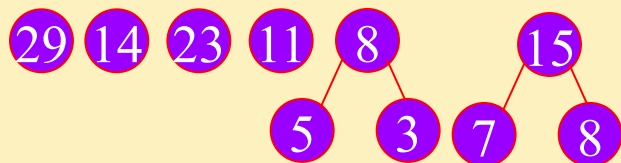
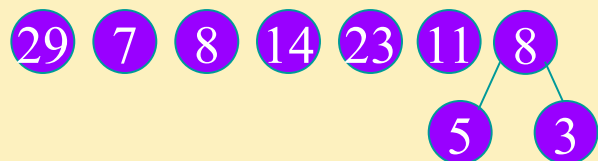
$$WPL = 2 \times 1 + 5 \times 3 + 7 \times 3 + 9 \times 3 + 6 \times 3 = 83 \quad (\text{右下图})$$

1.4.7 赫夫曼树及其应用

赫夫曼树的构造:

- (1) 根据给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$, 构造 n 棵二叉树的集合 $F = \{T_1, T_2, \dots, T_n\}$, 其中每棵二叉树中均只含一个带权值为 w_i 的根结点, 其左、右子树为空树;
- (2) 在 F 中选取其根结点的权值为最小的两棵二叉树, 分别作为左、右子树构造一棵新的二叉树, 并置这棵新的二叉树根结点的权值为其左、右子树根结点的权值之和; (根结点的权值 = 左右孩子权值之和, 叶结点的权值 = W_i) 。
- (3) 从 F 中删去这两棵树, 同时加入刚生成的新树;
- (4) 重复 (2) 和 (3) 两步, 直至 F 中只含一棵树为止。

例 $w=\{5, 29, 7, 8, 14, 23, 3, 11\}$



1.4.7 赫夫曼树及其应用

- 1. 判定树

在很多问题的处理过程中，需要进行大量的条件判断，这些判断结构的设计直接影响着程序的执行效率。

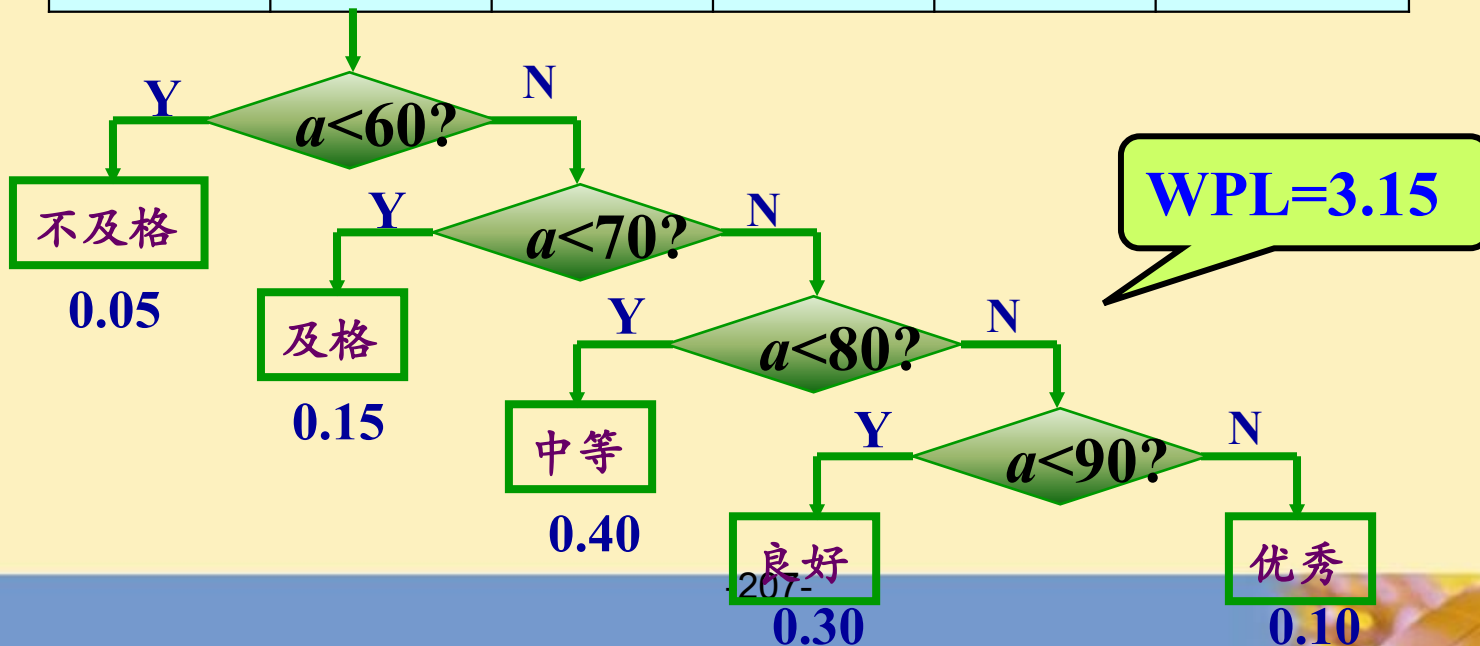
- 例如，编制一个程序，将百分制转换成五个等级输出。一种形式如下：

```
if (score<60) printf("bad");  
else if (score<70) printf("pass")  
else if (score<80) printf("general");  
else if (score<90) printf("good");  
else printf("very good");
```

1.4.7 赫夫曼树及其应用

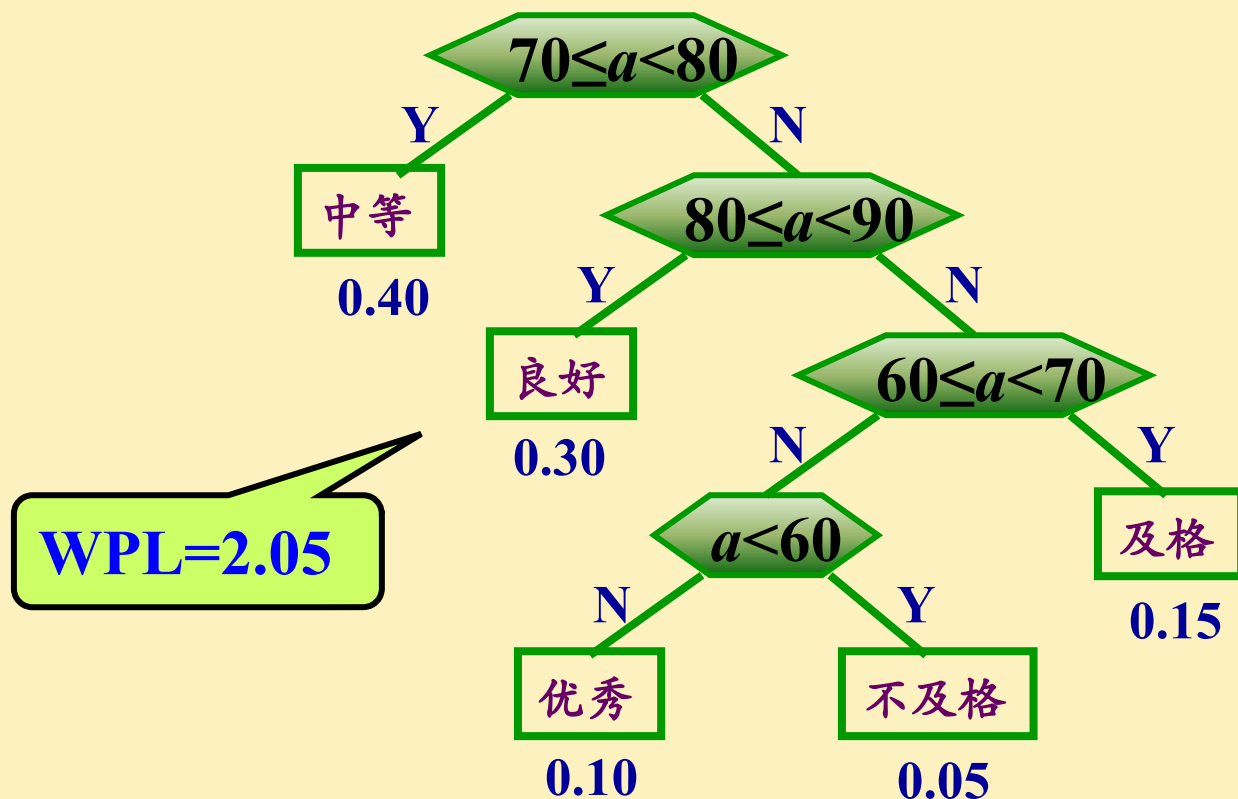
- 赫夫曼树的应用——最佳判定算法

分数	0~59	60~69	70~79	80~89	90~100
比例	0.05	0.15	0.40	0.30	0.10
等级	不及格	及格	中等	良好	优秀



1.4.7 赫夫曼树及其应用

- 赫夫曼树的应用——最佳判定算法



1.4.8 最短路径

- ❖ 我们研究带权图，一个重要的内容就是寻找某类**具有最小（最大）权的子图**，其中之一就是最短路问题，例如：给定一个连接各城市的铁路网络（连通的带权图），在这个网络中的两个指定的城市之间确定一条最短路。
- ❖ 所谓**最短路径**（shortest path）问题指的是：如果从图中某顶点出发（此点称为源点），经图的边到达另一顶点（称为终点）的路径不止一条，如何找到一条路径使沿此路径上各边的权值之和为最小。
- ❖ 设一有向网络 $G = (V, E)$ ，已知各边的权值，并设每边的权均大于零，以某指定 V_0 为源点，求从 V_0 到图的其余各点的最短路径。

1.4.8 最短路径

- 求最短路长的算法是 Dijkstra(狄克斯特拉) 于 1959 年提出来的, 是至今公认的求最短路长的最好算法, 称为 Dijkstra 算法。
- Dijkstra 算法功能: 在连通的带权图中, 求从 v_0 到 v 的最短路的路长。
- Dijkstra 算法基本思想: 将图 G 中结点集合 V 分成两部分, 一部分称为具有 P 标号的集合, 另一部分称为具有 T 标号的集合。
- 所谓结点 a 的 P 标号是指从 v_0 到 a 的最短路的路长, 而结点 b 的 T 标号是指从 v_0 到 b 的某条路径的长度。
- Dijkstra 算法中首先将 v_0 取为 P 标号结点, 其余的点均为 T 标号结点, 然后逐步地将具有 T 标号的结点改为 P 标号结点, 当结点 v 也被改为 P 标号时, 就找到了从 v_0 到 v 的最短路径的长度。

1.4.8 最短路径

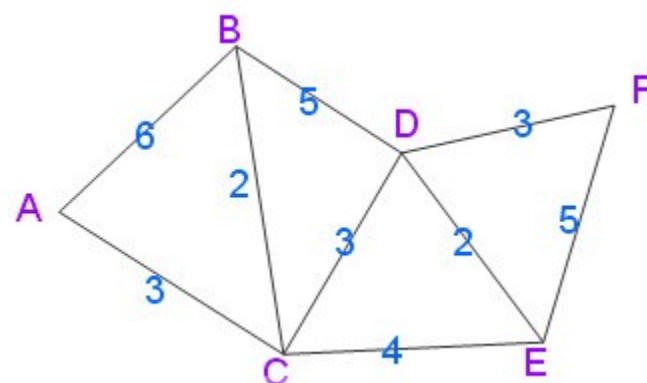
- Dijkstra 算法步骤:

(1) 初始时, S 只包含源点, 即 $S = v$ 的距离为 0。 U 包含除 v 外的其他顶点, v 到 U 中顶点 u 的距离为 v 到 u 边上的权。

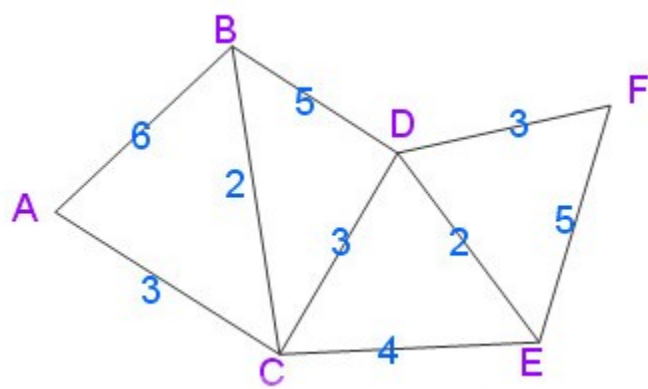
(2) 从 U 中选取一个距离 v 最小的顶点 k , 把 k 加入 S 中 (该选定的距离就是 v 到 k 的最短路径长度)。

(3) 以 k 为新考虑的中间点, 修改 U 中各顶点的距离; 若从源点 v 到顶点 u ($u \in U$) 的距离 (经过顶点 k) 比原来距离 (不经过顶点 k) 短, 则修改顶点 u 的距离值。

(4) 重复步骤 (2) 和 (3) 直到所有顶点都包含在 S 中。



步骤	S 集合中	U 集合中
1	选入 A，此时 $S = \langle A \rangle$ 此时最短路径 $A \rightarrow A = 0$ 以 A 为中间点，从 A 开始找	$U = \langle B, C, D, E, F \rangle$ $A \rightarrow B = 6$ $A \rightarrow C = 3$ $A \rightarrow \text{其他 } U \text{ 中的顶点} = \infty$ 发现 $A \rightarrow C = 3$ 权值为最短
2	选入 C，此时 $S = \langle A, C \rangle$ 此时最短路径 $A \rightarrow A = 0$ ， $A \rightarrow C = 3$ 以 C 为中间点，从 $A \rightarrow C = 3$ 这条最短路径开始找	$U = \langle B, D, E, F \rangle$ $A \rightarrow C \rightarrow B = 5$ （比上面第一步的 $A \rightarrow B = 6$ 要短） 此时到 B 权值为 $A \rightarrow C \rightarrow B = 5$ $A \rightarrow C \rightarrow D = 6$ $A \rightarrow C \rightarrow E = 7$ $A \rightarrow C \rightarrow \text{其他 } U \text{ 中的顶点} = \infty$ 发现 $A \rightarrow C \rightarrow B = 5$ 权值为最短
3	选入 B，此时 $S = \langle A, C, B \rangle$ 此时最短路径 $A \rightarrow A = 0$ ， $A \rightarrow C = 3$ ， $A \rightarrow C \rightarrow B = 5$ 以 B 为中间点，从 $A \rightarrow C \rightarrow B = 5$ 这条最短路径开始找	$U = \langle D, E, F \rangle$ $A \rightarrow C \rightarrow B \rightarrow D = 10$ （比上面第二步的 $A \rightarrow C \rightarrow D = 6$ 要长） 此时到 D 权值更改为 $A \rightarrow C \rightarrow D = 6$ $A \rightarrow C \rightarrow B \rightarrow \text{其他 } U \text{ 中的顶点} = \infty$ 发现 $A \rightarrow C \rightarrow D = 6$ 权值为最短



4	选入 D, 此时 $S = \langle A, C, B, D \rangle$ 此时最短路径 $A \rightarrow A = 0$, $A \rightarrow C = 3$, $A \rightarrow C \rightarrow B = 5$, $A \rightarrow C \rightarrow D = 6$ 以 D 为中间点, 从 $A \rightarrow C \rightarrow D$ 这条最短路径开始找	$U = \langle E, F \rangle$ $A \rightarrow C \rightarrow D \rightarrow E = 8$ (比上面第二步的 $A \rightarrow C \rightarrow E = 7$ 要长) 此时到 E 权值更改为 $A \rightarrow C \rightarrow E = 7$ $A \rightarrow C \rightarrow D \rightarrow F = 9$ 发现 $A \rightarrow C \rightarrow E = 7$ 权值为最短
5	选入 E, 此时 $S = \langle A, C, B, D, E \rangle$ 此时最短路径 $A \rightarrow A = 0$, $A \rightarrow C = 3$, $A \rightarrow C \rightarrow B = 5$, $A \rightarrow C \rightarrow D = 6$ $A \rightarrow C \rightarrow E = 7$ 以 E 为中间点, 从 $A \rightarrow C \rightarrow E = 7$ 这条最短路径开始找	$U = \langle F \rangle$ $A \rightarrow C \rightarrow E \rightarrow F = 12$ (比上面第四步的 $A \rightarrow C \rightarrow D \rightarrow F = 9$ 要长) 此时到 F 权值更改为 $A \rightarrow C \rightarrow D \rightarrow F = 9$ 发现 $A \rightarrow C \rightarrow D \rightarrow F = 9$ 权值为最短
6	选入 F, 此时 $S = \langle A, C, B, D, E, F \rangle$ 此时最短路径 $A \rightarrow A = 0$, $A \rightarrow C = 3$, $A \rightarrow C \rightarrow B = 5$, $A \rightarrow C \rightarrow D = 6$ $A \rightarrow C \rightarrow E = 7$, $A \rightarrow C \rightarrow D \rightarrow F = 9$	U 集合已空, 查找完毕。

2728areen-rock.blog.163.com